

Rapport technique — Projet EdgeSight

Détection embarquée, quantification INT8, suivi multi-objets et journalisation d'événements

10 septembre 2026

1 Introduction

EdgeSight est une démarche personnelle, construite autour d'un cas d'usage réaliste : des caméras autonomes à faible consommation d'énergie, destinées à des usages de sécurité, d'opérations et de suivi en temps réel. Le projet a été pensé pour couvrir deux compétences complémentaires — **IA Embarquée** et **IA Agentique** — qui se retrouvent directement dans les deux livrables visés : un modèle de détection optimisé pour l'embarqué, et un agent conversationnel capable d'interroger ses résultats en langage naturel. Le cahier des charges qui suit a été rédigé en amont, à la manière d'un brief client réaliste, pour ancrer les choix techniques dans des contraintes concrètes plutôt que dans un exercice académique sans limite.

1.1 Cahier des charges

Contraintes dures sur le modèle de détection :

- **full-CNN** — l'ensemble du modèle (backbone, cou, tête) doit être convolutif, sans composant hors CNN dans le chemin d'inférence
- **quantifié INT8** — le modèle final doit être livré quantifié, pas seulement entraîné en fp32
- **taille ≤ 5 Mo** une fois quantifié
- **IPS ≥ 25** — voir la note ci-dessous sur IPS vs FPS.

Et sur l'agent conversationnel : répondre à des requêtes en langage naturel portant sur les détections — compter les objets présents, filtrer par plage horaire, déclencher une alerte sur une durée de présence continue, ou sur le franchissement d'une zone, etc

Aucune contrainte matérielle (cible d'inférence, RAM disponible, CPU/GPU) n'est fixée : en l'absence de cible précise, la démarche retenue a été de mesurer les performances sur un poste de développement représentatif (CPU grand public), et de documenter les résultats comme des repères comparables plutôt que comme des garanties sur une cible embarquée réelle non spécifiée.

IPS (inférences par seconde) : ce rapport distingue volontairement IPS et FPS. Les chiffres de vitesse mesurés aux sections 4.3.1, 5.3–5.4 et 6 sont de l'**IPS** — la fréquence d'inférence pure du modèle seul (`benchmark.py`, section 6.1), hors capture vidéo, tracking et rendu. Le **FPS** réel d'une démonstration complète (section 10.2) est nettement inférieur. Ce travail d'optimisation hors modèle ne fait pas partie de ce projet.

1.2 Plan du rapport

Après ce chapitre d'introduction, la section 2 justifie le choix de l'architecture de détection. Les sections 3 et 4 couvrent la préparation des données et les trois générations successives de modèle entraîné. La section 5 détaille la toolchain de quantification INT8 développée pour ce

projet, et la section 6 en mesure les performances. La section 7 dresse un bilan honnête de la partie détection, limites comprises. La section 8 couvre le suivi multi-objets (tracking) et la journalisation des événements — développés avant l’agent conversationnel (section 9) car celui-ci s’appuie directement sur le journal produit ici. Les sections 9 et 10 couvrent respectivement l’agent conversationnel et l’intégration du système complet en une démonstration live. Le rapport se conclut par une synthèse générale et des pistes de prolongement (section 11).

2 Choix d’architecture de détection

2.1 Panorama des architectures candidates

Modèle	Full CNN	Graphe propre	exporté	Code source transparent	Poids entraînés pendants	pré-indé-	Taille fp32 estimée ²	Taille INT8 estimée ³	mAP COCO 80 classes
YOLOX-Nano	Oui	Oui — décodage/NMS hors graphe	Bon		Oui		3,6 Mo	0,9 Mo	~25,8%
YOLOX-Tiny	Oui	Oui (même famille)	Bon		Oui		20,2 Mo	5,1 Mo — limite	~32,8%
NanoDet-Plus-m (1.0x), 320	Oui	Oui — décodage/NMS hors graphe	Très bon		Oui		4,7 Mo	1,2 Mo	~27,0%
NanoDet-Plus-m (1.0x), 416	Oui	Oui (même famille)	Très bon		Oui		4,7 Mo	1,2 Mo	~30,4%
NanoDet-Plus-m-1.5x, 320	Oui	Oui (même famille)	Très bon		Oui		9,8 Mo	2,4 Mo	~29,9%
NanoDet-Plus-m-1.5x, 416	Oui	Oui (même famille)	Très bon		Oui		9,8 Mo	2,4 Mo	~34,1%
YOLOv5n	Oui	Oui — décodage/NMS hors graphe	Bon (Ultralytics)		Oui		7,6 Mo	1,9 Mo	28,0%
YOLOv8n	Oui	Oui — décodage/NMS hors graphe	Bon (Ultralytics)		Oui		12,8 Mo	3,2 Mo	37,3%
PP-PicoDet-XS	Oui	Partiel — écosystème PaddlePaddle	Bon, mais hors PyTorch		Oui		2,8 Mo	0,7 Mo	23,5–26,2% ¹
MobileNetV3-Large + SSDLite	Oui	Oui — NMS hors graphe	Très bon (torchvision)		Oui		13,8 Mo	3,4 Mo	~22,3%
EfficientDet-Lite0	Oui	Non — export natif TFLite	Bon, mais hors PyTorch/ONNX		Oui		12,8 Mo	3,2 Mo	25,7%

¹ *PicoDet-XS : mAP officiel COCO val (README PaddleDetection, branche release/2.7) — 23,5% à 320px, 26,2% à 416px, pour 0,70M de paramètres.*

² *Tailles fp32 dérivées du nombre de paramètres officiellement publié par chaque projet ($\times 4$ octets/paramètre) — pas d’un fichier téléchargé et pesé, pour rester comparables entre frameworks qui n’empaquent pas leurs poids de la même façon (certains fichiers officiels contiennent en plus l’état de l’optimiseur, une copie EMA, ou le graphe).*

³ *Taille INT8 = fp32 $\div$ 4 (1 octet/paramètre) — une première approximation, pas une mesure de quantification réelle.*

Les tailles fp32/INT8 de ce tableau sont des **estimations a priori** (nombre de paramètres officiellement publié $\times$ taille d’un flottant/entier), établies avant tout export réel — une **première approximation**, volontairement simple pour rester comparable entre candidats de frameworks différents, pas une mesure de taille de fichier réelle. La taille estimée en INT8 sera confrontée au résultat mesuré une fois le modèle retenu effectivement quantifié (section 5.6).

2.2 Critères de sélection

- **Full CNN** — contrainte dure du cahier des charges imposé : l’architecture retenue doit être entièrement convolutive.
- **Graphe exporté propre** — on veut pouvoir exporter uniquement la passe avant (forward) du réseau, sans figer dans le graphe ONNX des artéfacts comme le décodage des boîtes ou le NMS (non-maximum suppression). Ces étapes restent gérées en dehors du graphe, dans du code que l’on contrôle entièrement — condition nécessaire pour appliquer notre propre toolchain de quantification sans avoir à quantifier des opérateurs de post-traitement non conçus pour ça.

- **Code source transparent** — le modèle doit être facilement manipulable : configuration lisible, architecture modifiable (adapter le nombre de classes, l’ajout d’une résolution d’entrée différente).
- **Poids pré-entraînés sur COCO** — (voir intro section 3 pour justification du dataset COCO) la disponibilité de poids pré-entraînés sur ce corpus, publiés indépendamment de l’entraînement qu’on va mener, permet de partir d’un fine-tuning plutôt que d’un entraînement from scratch.
- **Taille fp32 estimée** — poids du modèle avant quantification (estimation via le nombre de paramètres) ; sert de point de comparaison et permet d’avoir une idée de la difficulté de le faire tenir dans 5 Mo en INT8.
- **Taille INT8 estimée** — indicateur direct de respect de la contrainte ≤ 5 Mo, avant même d’avoir construit la toolchain de quantification.
- **mAP COCO 80 classes** — (voir intro section 3 pour justification du dataset COCO) seul signal de qualité disponible à ce stade : aucun de ces modèles n’a été évalué sur une tâche à 2 classes (person/car) avant notre propre entraînement, donc le mAP publié sur les 80 classes de COCO sert de proxy comparatif entre candidats (voir 2.4 pour la discussion de sa transférabilité).

Deux critères supplémentaires ont motivé l’exclusion de certains modèles, malgré des chiffres parfois compétitifs :

- **Écosystème PyTorch natif** — la toolchain de quantification développée pour ce projet (section 5) s’appuie directement sur `torch.onnx.export` depuis des poids PyTorch. PP-PicoDet (PaddlePaddle) et EfficientDet-Lite (TensorFlow/TFLite) auraient demandé soit une conversion cross-framework supplémentaire (Paddle2ONNX), soit un réentraînement dans un framework différent — un coût d’intégration disproportionné pour un projet où la valeur ajoutée porte sur la toolchain de quantification, pas sur l’architecture elle-même.
- **Licence compatible avec un usage en entreprise** — YOLOv5n et YOLOv8n (Ultralytics) sont publiés sous licence **AGPL-3.0** : sans licence commerciale payante, toute réutilisation en contexte entreprise oblige à publier l’intégralité du code source du projet qui les intègre. C’est un motif d’exclusion à part entière, y compris pour YOLOv8n dont le mAP publié (37,3%) dépasse celui du modèle finalement retenu (34,1%) — un critère non technique qui l’emporte ici sur la seule performance brute.

2.3 Modèle retenu et justification

Architecture retenue : NanoDet-Plus-m-1.5x, résolution 416 (backbone ShuffleNetV2, cou Ghost-PAN, tête anchor-free de type GFL).

Parmi les deux finalistes respectant confortablement la contrainte de taille une fois quantifiés (NanoDet-Plus-m 1.0x et 1.5x, tous deux à 416px, $\sim 1,2$ Mo et $\sim 2,4$ Mo estimés), le choix s’est porté sur la variante **1.5x** plutôt que sur la plus petite des deux. La contrainte des 5 Mo est un plafond binaire à respecter, pas un objectif à minimiser au-delà de ce qui est demandé : une fois cette contrainte largement respectée par les deux candidats, le seul delta réellement différenciant entre eux est la précision ($\sim 30,4\%$ contre $\sim 34,1\%$ de mAP COCO 80 classes) — un gain qui se repercutera à priori sur les performances finales du modèles après fine-tuning sur 2 classes.

2.4 Transférabilité du mAP 80 classes vers 2 classes (person/car)

Le mAP publié (34,1%) est mesuré sur les 80 classes de COCO — la question posée avant tout entraînement était de savoir si ce chiffre était représentatif de la performance attendue sur notre tâche à 2 classes seulement (person/car). Deux arguments qualitatifs suggéraient un score réel supérieur au chiffre publié :

- **person** et **car** sont des classes réputées « faciles » par rapport à la moyenne de COCO — le mAP 80-classes est tiré vers le bas par des catégories rares ou visuellement ambiguës, alors que **person** (la classe la mieux représentée dans COCO) obtient typiquement un AP par classe 1,3 à 1,6 fois le mAP moyen d’un modèle, et **car** autour de 1,0 à 1,3 fois ;
- ramener la tâche à 2 classes (contre 80) simplifie mécaniquement la partie classification du problème, sans changer la difficulté de localisation des boîtes elle-même.

Une réserve inverse doit cependant être posée dès maintenant : **car**, dans les scènes COCO, est une classe fréquemment **petite ou photographiée à distance** (véhicules en arrière-plan d’une rue, d’un parking, d’une scène urbaine large) — une difficulté de **localisation**, pas de classification, que le passage à 2 classes ne résout donc pas. Or la résolution d’entrée choisie en section 2.3 (416px) reste un choix par défaut, hérité du modèle candidat retenu, pas un choix validé pour ce projet en particulier. Cette réserve motive directement la démarche de la section 4 : tester empiriquement, sur plusieurs générations successives, si une résolution d’entrée plus fine profite spécifiquement à **AP_small** (et donc, en pratique, en bonne partie à **car**).

3 Préparation des données

Pourquoi COCO : COCO fournit, gratuitement et à grande échelle, les deux classes recherchées (**person**, **car**), avec des annotations et une évaluation déjà standardisées (**pycocotools**) directement réutilisées telles quelles dans ce projet (sections 5, 6) — un gain d’implémentation réel. C’est aussi un corpus suffisamment connu et documenté pour que les choix qui en découlent restent vérifiables par un lecteur externe, plutôt qu’un jeu de données maison invérifiable.

Ce choix, une fois posé, **motive à son tour** un des critères de sélection d’architecture (section 2.2) : chercher un modèle disposant déjà de poids pré-entraînés *spécifiquement sur COCO* (plutôt que sur un autre corpus, ou aucun pré-entraînement) devient naturel dès lors que COCO est le corpus cible retenu — la dépendance va donc de la donnée vers l’architecture, pas l’inverse. Fine-tuner sur ce même corpus, plutôt que de basculer vers un dataset différent une fois l’architecture choisie (images de sécurité collectées spécifiquement, BDD100K, Open Images...), minimise par ailleurs le décalage de distribution (*domain shift*) entre pré-entraînement et fine-tuning — un facteur connu pour affecter la vitesse de convergence, parfois le résultat final, d’un transfert d’apprentissage.

Contrepartie assumée, non mesurée ici : COCO reste un corpus de scènes générales (photographies grand public, prise à hauteur d’homme) — un vrai écart de domaine avec le cas d’usage visé peut donc se créer, ce qui n’a pas été quantifié empiriquement dans ce projet (voir limites, section 7). Par ailleurs les vidéos de démonstrations du projet ne sont pas issues du dataset COCO et peuvent donc subir cet écart de domaine.

3.1 Source et dimensionnement du jeu de données

Les données proviennent de **COCO 2017** (**train2017** et **val2017**), filtrées aux deux seules classes qui nous intéressent : **person** et **car**. Ce dataset est **déséquilibré par nature** : certaines catégories y sont structurellement sur-représentées par rapport à d’autres, et cette sur-représentation joue ici en faveur de **person** face à **car**. L’échantillonnage (**detection/src/01_data/data_filter**) répartit les images candidates en trois catégories mutuellement exclusives selon leur contenu — **only_person** (uniquement des personnes), **only_car** (uniquement des voitures) et **both** (les deux classes présentes).

Split	pool person-only	pool car-only	pool both
train2017	55 596 img	3 732 img	8 519 img
val2017	2 334 img	176 img	359 img

Cette inspection oriente directement la stratégie retenue. Un quota unique appliqué identiquement aux trois catégories a été écarté d’emblée : il traiterait **car-only** (pool réduit, 3 732 images) et **person-only** (pool quasi illimité, 55 596 images) de façon symétrique, alors qu’ils n’ont rien de comparable — un quota qui semblerait raisonnable pour l’un serait soit trivial, soit un plafond arbitraire pour l’autre. La stratégie retenue prend l’**intégralité** des pools **car-only** et **both** (3 732 et 8 519 images), et plafonne **only_person** à **5 000** — volontairement bien en dessous du pool réel disponible (55 596) — précisément pour ne pas diluer davantage la part de **car** dans le jeu de données final.

Contrairement à **train2017**, le pool **val2017** ainsi constitué est ensuite scindé en deux : **val** et **test**. Cette séparation répond à un besoin méthodologique précis, indépendant du déséquilibre décrit ci-dessus : COCO ne publie pas les annotations de son propre split **test2017** (réservées aux serveurs d’évaluation officiels), donc aucune mesure locale de mAP n’y est possible. Or deux rôles distincts doivent être tenus par des données *différentes* : **val** sert à sélectionner le meilleur checkpoint pendant l’entraînement (**model_best**, section 4), tandis que **test** sert exclusivement à la mesure finale rapportée. Utiliser le même jeu de données pour les deux rôles biaiserait ce chiffre final de façon optimiste : le choix même du checkpoint serait une décision informée par ces données, ce qui reviendrait à évaluer le modèle sur des données qui ont indirectement influencé sa sélection. Le split est réalisé par tirage aléatoire (`sklearn.model_selection.train_test_split`, `test_size=0,3`, graine fixée à 42 pour la reproductibilité) sur les images **val2017** déjà retenues ci-dessus — sans stratification explicite par catégorie, un choix simple qui n’a pas été poussé plus loin. **test** représente ainsi 30% de ce pool, et n’est jamais vu ni pendant l’entraînement ni pendant la sélection du checkpoint (section 4).

Volumes résultants :

Split	Images	Annotations person	Annotations car
train	17 251	64 628	43 867
val	2 008	7 546	1 321
test	861	3 458	611

Les trois pools de **val2017** sont déjà plus petits que les cibles retenues pour l’entraînement : le split **val/test** utilise donc déjà **100%** des images disponibles pour **car**. Le déséquilibre observé sur ces splits (nettement plus d’images **person** que **car**) est ainsi **inhérent à COCO val2017 lui-même**, pas un artefact de notre échantillonnage — il ne peut donc pas être corrigé par ce levier, et la mesure de performance sur **car** y reste intrinsèquement plus bruitée (volume d’annotations plus faible). Ce bruit reste toutefois probablement d’une ampleur limitée à cette échelle : la variance d’une métrique agrégée comme le mAP décroît avec le nombre d’annotations — quelques centaines d’annotations suffisent déjà à produire une estimation raisonnablement stable, et rien ne garantit qu’évaluer sur 10 000 annotations **car** plutôt que 611 changerait qualitativement la conclusion.

3.2 Asymétrie résiduelle assumée

Même avec cette stratégie — prendre l’intégralité de ce qui existe pour **car**, plafonner **person-only** bien en dessous de son pool réel — le jeu de données d’entraînement final conserve davantage d’annotations **person** (64 628) que **car** (43 867) : ratio $\sim 1,47$. Ce résidu n’est pas un signe d’échantillonnage insuffisant, mais une contrainte structurelle de COCO lui-même : même en exploitant à 100% ce qui existe pour **car** (**car-only** + **both**, 3 732 + 8 519 = 12 251 images), ce total reste plus petit que le seul pool **person-only** (55 596 images) dont on ne prélève pourtant qu’une fraction (5 000) — un déséquilibre qui persiste même après un échantillonnage pensé pour le minimiser.

Conséquence attendue, détaillée en section 4 : les modèles entraînés sur ce jeu de données reconnaîtront à priori un peu mieux **person** que **car** (1.47x plus d’annotations d’en-

entraînement **person** que **car**) — à comparer avec le ratio $\text{mAP person} / \text{mAP car}$ à l'issues des entraînements, pour voir si ces dernier arrivent à combler l'asymétrie des données d'entraînements.

4 Entraînement — trois générations de modèle

Le modèle a été entraîné en trois générations successives, **sur le même jeu de données** (section 3), chacune testant une résolution d'entrée plus élevée que la précédente. Cette progression est motivée par une hypothèse posée dès la section 2.4 : les voitures, souvent petites ou photographiées à distance dans les scènes COCO, devraient bénéficier directement d'une résolution d'entrée plus fine — une résolution plus grande aide mécaniquement la détection des petits objets. Cette hypothèse a pu être confirmé empiriquement avec l'entraînement de génération 2, puis poussé à sa limite sur un dernier entraînement de génération 3 (voir section 4.3.1). Le matériel utilisé pour l'ensemble des entraînements est un GPU portable RTX 4060 (8 Go de VRAM), 12 cœurs CPU.

4.1 Génération 1 — baseline 416px

4.1.1 Paramètres

Le fichier de configuration NanoDet-Plus, dupliqué depuis le dépôt d'origine, contenait encore des valeurs par défaut pensées pour un entraînement *from scratch* sur COCO complet (118k images, 300 epochs, cluster multi-GPU) — inadaptées à un fine-tuning sur 17 251 images avec un seul GPU portable. Les principaux ajustements sont les suivants :

Paramètre	Hérité	Retenu	Justification
<code>batchsize_per_gpu</code>	96	24	Calibré pour un GPU datacenter (24–80 Go) ; 8 Go de VRAM impose une réduction forte, ajustée empiriquement
<code>precision</code>	32	16 (AMP)	Tensor cores disponibles sur la RTX 4060 — réduit la VRAM et accélère l'entraînement, quasi gratuit en qualité
<code>optimizer.lr</code>	0,001	0,00025	Fine-tuning depuis un modèle déjà bon $\Rightarrow$ LR prudent, cohérent avec la règle de mise à l'échelle linéaire <code>batch/lr</code>
<code>warmup.steps</code>	500	200	Dataset 10× plus petit que COCO complet, départ depuis des poids déjà entraînés
<code>total_epochs</code>	300	50	Aligné sur le budget retenu pour les deux autres générations (section 4.2/4.3), pour neutraliser le nombre d'epochs comme facteur de confusion dans la comparaison entre résolutions
<code>val_intervals</code>	10	4	Avec 50 epochs, valider tous les 10 epochs ne donnerait que 5 points de mesure — trop grossier

4.1.2 Courbes de loss

Le plateau s'est confirmé à l'**epoch 20** (`model_best`, $\text{mAP val}=0,382$). La loss de validation décroît puis se stabilise en suivant d'assez près la loss d'entraînement, sans écart anormal.

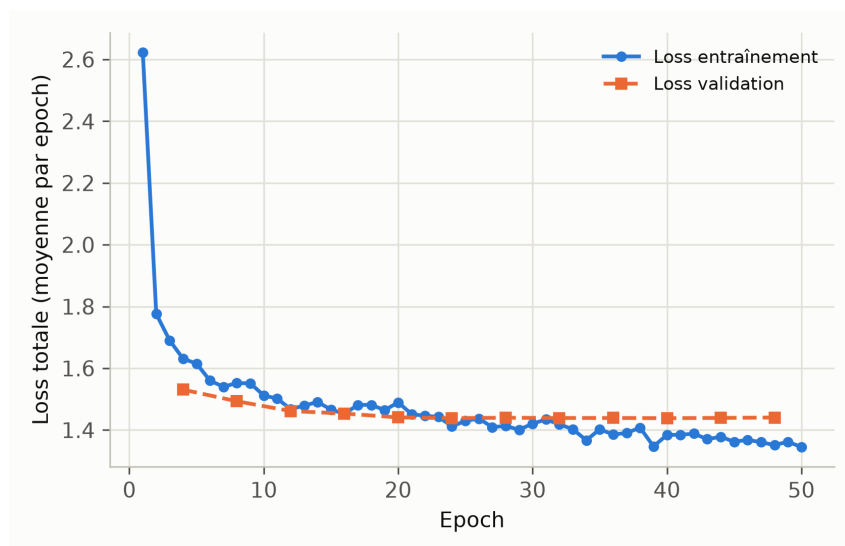


FIGURE 1 – Loss d'entraînement vs validation par epoch, génération 1 (416px).

4.1.3 Courbes de précision

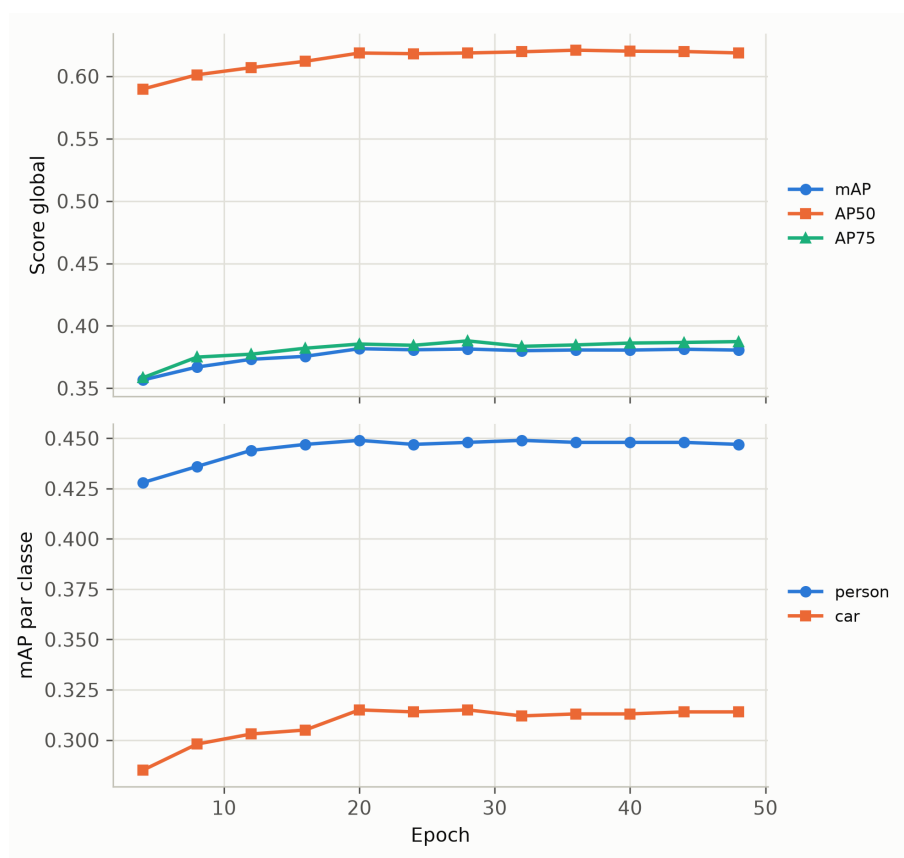


FIGURE 2 – Métriques de détection par epoch de validation, génération 1 (416px) — score global (mAP/AP50/AP75) et mAP par classe.

4.1.4 Précision du meilleur modèle

Résultat sur `test.json` (fp32) :

Métrique	416px fp32
mAP	0,382
AP50	0,618
AP75	0,390
AP_small	0,175
AP_medium	0,540
AP_large	0,712

On observe comme attendu un score nettement moins bon pour les petits objets (`AP_small`) du à la faible résolution d'entrée.

4.2 Génération 2 — passage à 512px

4.2.1 Paramètres

Motivée par l'hypothèse de la section 2.4, cette génération bénéficie d'une résolution d'entrée de 512px, soit un gain de 151% pixels.

Les hyperparamètres ont été réajustés selon la même règle de mise à l'échelle proportionnelle batch/LR utilisée en génération 1 :

Paramètre	416px (gén. 1)	512px (gén. 2)	Justification
<code>input_size</code>	[416,416]	[512,512]	Teste directement l'hypothèse posée en section 2.4 sur les petits objets
<code>batchsize_per_gpu</code>	24	16	Réduit proportionnellement ($24/1,51 \approx 16$) pour rester dans le même budget VRAM à $1,51 \times$ plus de pixels/image
<code>optimizer.lr</code>	0,0001	0,00007	Règle de mise à l'échelle linéaire batch/lr appliquée au nouveau batch
<code>warmup.steps</code>	200	300	Prudence supplémentaire : le réseau doit reprendre le fine-tuning <i>et</i> s'adapter à une résolution d'entrée différente

`total_epochs` (50) et `val_intervals` (4) restent **inchangés** par rapport à la génération 1 — avec le jeu de données désormais constant entre les trois générations (section 3), la résolution d'entrée et les hyperparamètres qui en découlent mécaniquement (batch, LR, warmup) sont la **seule** chose qui diffère d'une génération à l'autre.

4.2.2 Courbes de loss

Le plateau s'est confirmé à l'**epoch 36** (`model_best`, mAP val=0,4165), avec une convergence stable en fin d'entraînement — pas de recul marqué.

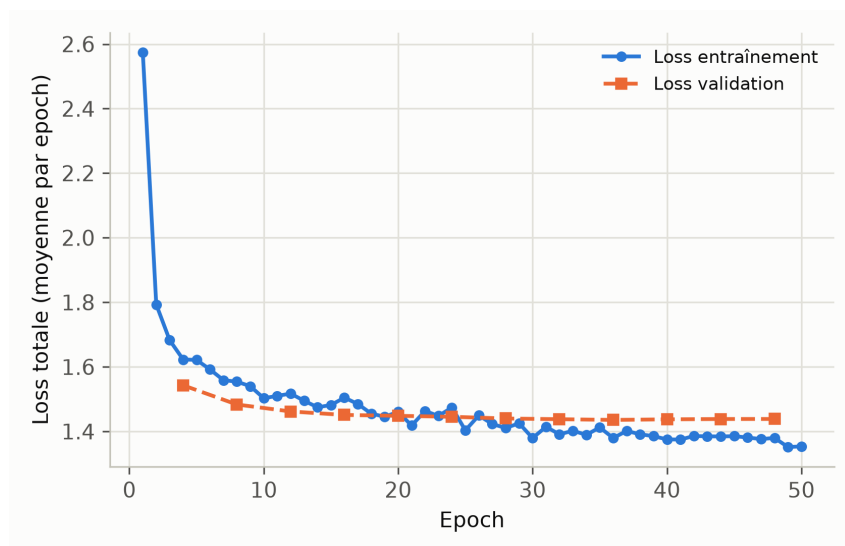


FIGURE 3 – Loss d'entraînement vs validation par epoch, génération 2 (512px)

4.2.3 Courbes de précision

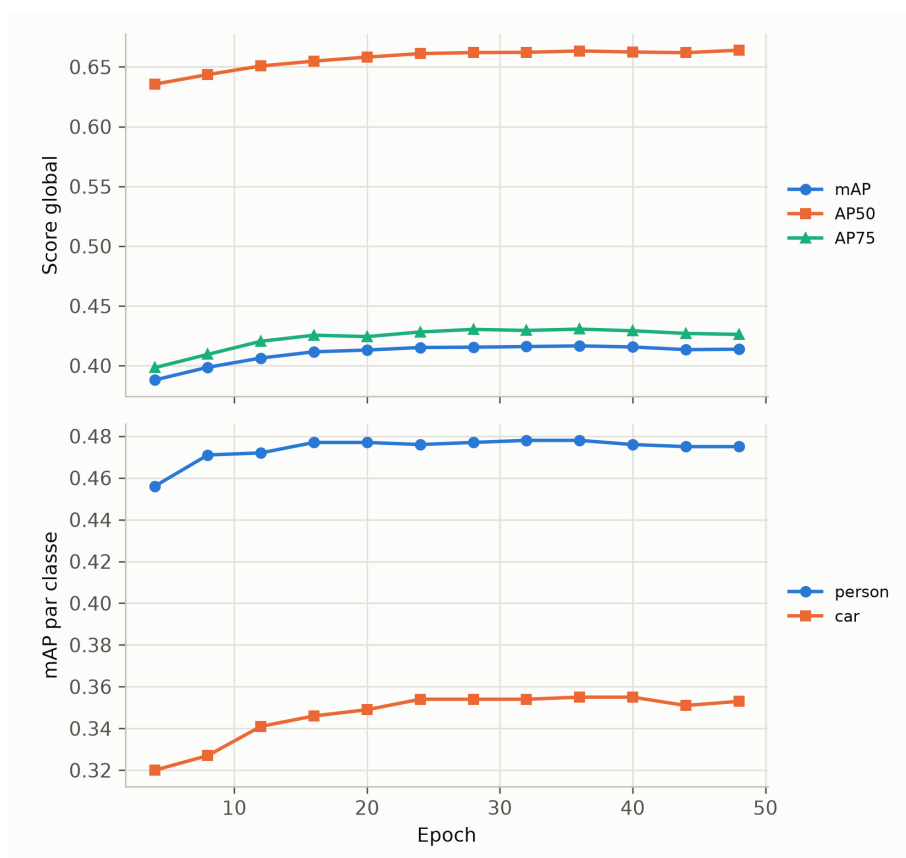


FIGURE 4 – Métriques de détection par epoch de validation, génération 2 (512px) — score global (mAP/AP50/AP75) et mAP par classe.

4.2.4 Précision du meilleur modèle

Résultat sur `test.json` (fp32) :

Métrique	512px fp32	vs 416px
mAP	0,421	+10,2%
AP50	0,668	+8,1%
AP75	0,438	+12,3%
AP_small	0,228	+30,3%
AP_medium	0,583	+8,0%
AP_large	0,712	+0,0%

On observe le plus gros gain relatif sur la métrique **AP_small**, augmentant de plus de 30% ce qui semble confirmer l’hypothèse posée en section 2.4. Globalement cette résolution supérieur profite à toute les métrique à l’exception de **AP_large** qui n’évolue pas du tout (0,712 dans les deux générations) — cohérent avec l’idée que le gain de résolution profite spécifiquement aux petits objets, pas aux grands qui étaient déjà bien détectés.

4.3 Génération 3 — recherche du plafond de résolution, 896px

4.3.1 Paramètres

Le passage à 512px ayant confirmé l’hypothèse de départ (**AP_small** en nette progression au fil de l’entraînement, section 4.2), la question naturelle était de savoir jusqu’où pousser la résolution avant que le compromis IPS ne devienne défavorable. Point méthodologique notable : la **latence d’un modèle ne dépend que de sa résolution et de son architecture, pas des poids entraînés**. Il est donc possible de mesurer le IPS à plusieurs résolutions candidates en export/quantifiant le modèle 512px *déjà entraîné* à ces résolutions, **sans réentraîner** — et donc sans engager des heures de calcul avant de savoir si la résolution visée est viable en pratique. Mesures effectuées sur le **candidat de déploiement** (ONNX INT8 QDQ u8s8, CPU — section 5.3), avec la même méthodologie que `benchmark.py` (section 6.1).

Résolution	Latence médiane	IPS	P95	ΔMém
416	10,22 ms	97,8	13,02 ms	14,08 Mo
512	13,67 ms	73,2	16,85 ms	20,04 Mo
640	19,36 ms	51,6	22,40 ms	24,79 Mo
768	26,73 ms	37,4	30,13 ms	44,29 Mo
832	31,33 ms	31,9	36,20 ms	47,67 Mo
896	35,53 ms	28,1	41,38 ms	56,41 Mo
960	40,48 ms	24,7	45,94 ms	74,91 Mo
1024	46,71 ms	21,4	53,51 ms	86,53 Mo

Le seuil de ≥ 25 IPS retenu (§1.1) est un **plancher** à respecter, pas un objectif à dépasser largement — le même type de critère que la taille maximale en section 2.3. Sur les huit résolutions mesurées ci-dessus, **896px est le dernier palier qui respecte encore ce plancher** (28,1 IPS, $\sim 12\%$ de marge) : **960px repasse sous le seuil** (24,7 IPS, en inférence pure seule — avant même le reste du pipeline réel : décodage boîtes/NMS, tracker, affichage, qui ne fait qu’ajouter de la latence), et 1024px est nettement en dessous.

896px est donc retenu directement comme résolution de la génération 3 — un saut de résolution substantiel vs 512px ($\times 3,06$ en pixels), au prix d’un coût de calcul en conséquence (~ 31 h de calcul actif, réparties en deux segments séparés par une pause manuelle de 48 heures après l’epoch 35 — section 4.3.2).

Les hyperparamètres ont de nouveau été réajustés proportionnellement :

Paramètre	512px (gén. 2)	896px (gén. 3)	Justification
input_size	[512,512]	[896,896]	Résolution retenue ci-dessus
batchsize_per_gpu	16	6	$16/(896/512)^2 \approx 5,2$, arrondi à 6
optimizer.lr	0,00007	0,00002625	$\times(6/16)$, même règle de mise à l'échelle
warmup.steps	300	450	Prudence supplémentaire, saut de résolution plus grand qu'en génération 2

4.3.2 Courbes de loss

Le plateau final se confirme à l'**epoch 30** (`model_best`, mAP val=0,4813).

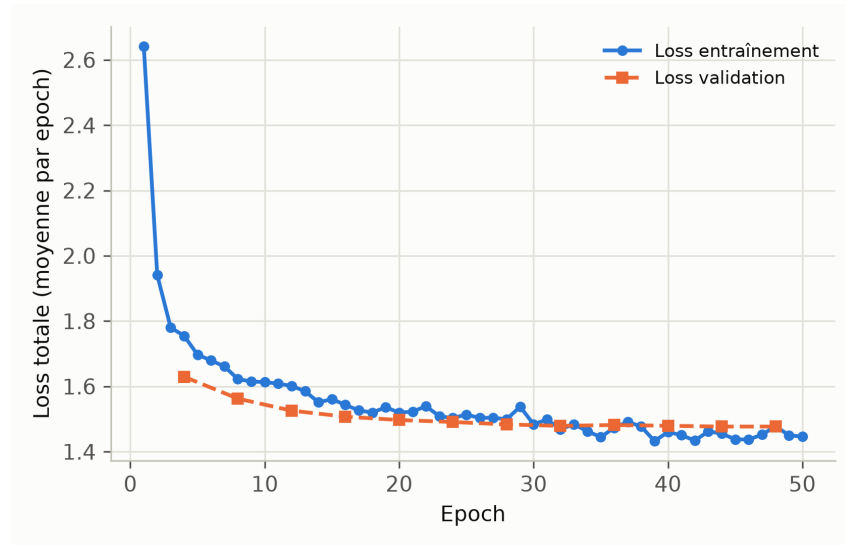


FIGURE 5 – Loss d'entraînement vs validation par epoch, génération 3 (896px). Convergence sans décrochage, aucun signe de sur-apprentissage sur cette métrique ; les deux courbes restent proches d'un bout à l'autre, comme pour les autres générations (section 4.1).

4.3.3 Courbes de précision

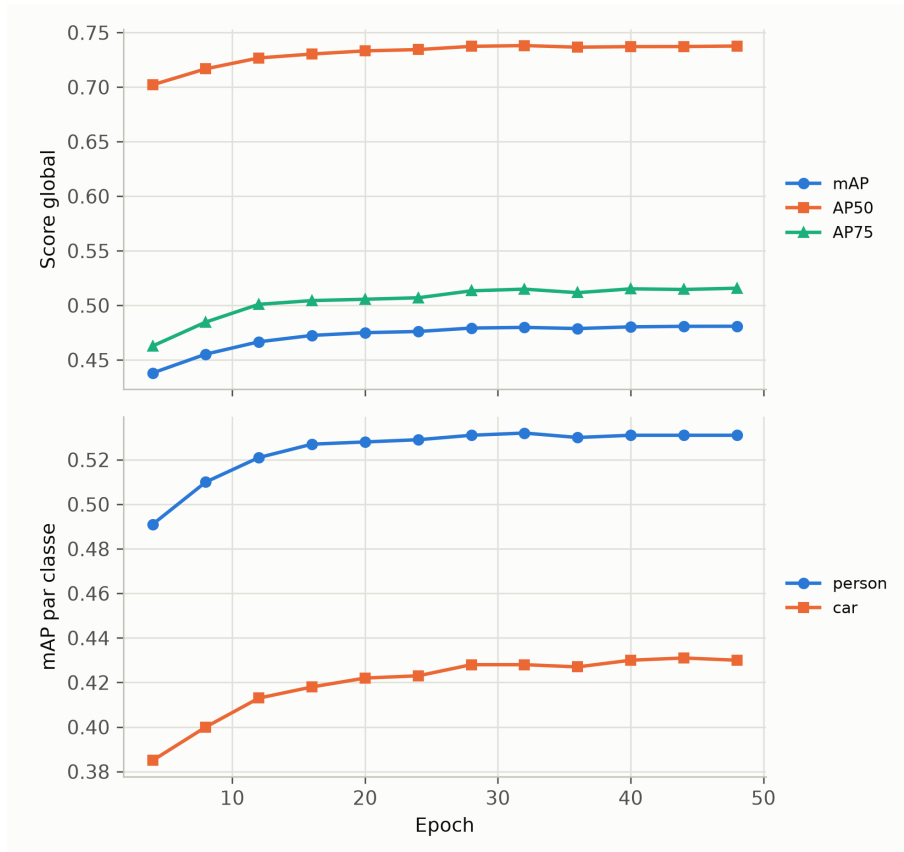


FIGURE 6 – Métriques de détection par epoch de validation, génération 3 (896px) — score global (mAP/AP50/AP75) et mAP par classe. Progression monotone jusqu’au plateau (~epoch 30), sans recul notable ensuite.

4.3.4 Précision du meilleur modèle

Résultat final sur `test.json` (fp32) :

Métrique	896px fp32	vs 512px	vs 416px
mAP	0,485	+15,2%	+27,0%
AP50	0,739	+10,6%	+19,6%
AP75	0,509	+16,2%	+30,5%
AP_small	0,335	+46,9%	+91,4%
AP_medium	0,629	+7,9%	+16,5%
AP_large	0,684	-3,9%	-3,9%

Seule `AP_large` recule (-3,9%) — cohérent avec une hypothèse plausible (la résolution redistribue une partie de la capacité du modèle vers les petits objets, un peu au détriment des grands) mais non vérifiée plus finement ici, sans que ça remette en cause le gain global. Sur l’ensemble de la progression 416px→896px, `AP_small` cumule un gain de **+91,4%** — de loin la métrique la plus transformée par la hausse de résolution, cohérent de bout en bout avec l’hypothèse posée dès la section 2.4.

4.4 Bilan de progression

Comparaison des meilleurs modèles (`model_best`) des trois générations, fp32, sur `test.json` — mêmes 6 métriques que les tableaux de précision de chaque génération (sections 4.1–4.3) :

Métrique	416px (gén. 1)	512px (gén. 2)	896px (gén. 3)
mAP	0,382	0,421	0,485
AP50	0,618	0,668	0,739
AP75	0,390	0,438	0,509
AP_small	0,175	0,228	0,335
AP_medium	0,540	0,583	0,629
AP_large	0,712	0,712	0,684

Ratio `person:car` (mAP, `model_best`) des trois générations, à comparer au ratio brut des annotations du jeu de données (1,47, section 3.2) :

	416px (gén. 1)	512px (gén. 2)	896px (gén. 3)
person (mAP)	44,8%	47,8%	53,2%
car (mAP)	31,2%	35,7%	43,1%
ratio person :car	1,44	1,34	1,23

Le ratio mesuré sur les modèles (1,44 puis 1,34 puis 1,23) reste à chaque génération **inférieur** au ratio brut des annotations du jeu de données (1,47), et se resserre de façon monotone sur les 3 générations — l’entraînement ne se contente pas d’hériter du déséquilibre des données, il le compense partiellement, et ce gain s’accroît avec la résolution plutôt que de stagner. Le déséquilibre n’est cependant jamais éliminé (le ratio reste >1 à chaque génération).

5 Quantification INT8 — toolchain maison

5.1 Pipeline général

La quantification post-entraînement (PTQ) est développée entièrement en interne, sans dépendre d’un outil clé-en-main : **export ONNX** → **validation numérique** → **calibration** → **quantification PTQ** → **validation taille/précision**. Chaque étape est un script Python autonome (`export_onnx.py`, `validate_onnx.py`, `quantize.py`, `evaluate_int8.py`), détaillés dans les sous-sections suivantes.

5.2 Export et validation

Le graphe est exporté avec `torch.onnx.export`, puis simplifié avec `onnxsim` et validé structurellement avec `onnx.checker`. La liste des opérateurs présents dans le graphe final est inspectée explicitement : **aucun opérateur de NMS n’apparaît** — confirmation directe, sur le graphe réellement exporté et non plus seulement sur la documentation de l’architecture, du critère « graphe exporté propre » posé en section 2.2.

La fidélité numérique de l’export est vérifiée sur 3 images de test réelles, en comparant la sortie brute PyTorch (fp32) à la sortie ONNX sur le même tenseur d’entrée : écart maximal mesuré $1,1\text{--}1,7 \times 10^{-5}$ (tolérances standard `rtol=1e-3`, `atol=1e-5`) — l’export ne dégrade pas la précision du modèle.

5.3 Choix du format d’export et du type d’activation

Avant même de choisir une méthode de calibration, deux paramètres de `quantize_static` restent à trancher : le format d’export et le type des activations. Toutes les mesures de cette section sont faites sur le **modèle final 896px**, pour rester directement comparables au modèle effectivement déployé plutôt qu’à des résolutions intermédiaires depuis retirées, avec une calibration `MinMax` tenue constante pour isoler ces deux seules variables.

5.3.1 Type des activations : u8s8 contre s8s8

Deux combinaisons possibles pour le signe des poids/activations quantifiés :

- **s8s8** — poids et activations tous deux en int8 signé — le choix qui semblait le plus simple au départ.
- **u8s8** — activations en uint8 non signé, poids en int8 signé.

Mesuré au banc (`benchmark.py`, warmup 15, 150 mesures, format QDQ tenu constant pour isoler cette seule variable) :

QDQ	s8s8	u8s8
Latence médiane	90,4 ms	31,4 ms
IPS	11,1	31,8

Le format **s8s8** rend l'INT8 nettement plus lent que le format **u8s8**. Après étude de la documentation, il s'avère que sur CPU x86-64 avec extensions AVX2/AVX512, `onnxruntime` exécute la combinaison **u8×s8** via l'instruction SIMD dédiée `VPMADDUBSW` (remplacée par l'instruction native `VNNI` sur les CPU qui la supportent, sans le risque de saturation propre à `VPMADDUBSW` sur AVX2/AVX512 seul) — un chemin de calcul rapide qui n'a pas d'équivalent pour **s8×s8**, d'où le repli sur un chemin générique bien plus lent¹.

5.3.2 Format d'export : QDQ contre QOperator

Ici aussi, deux choix candidats retenus, avec **u8s8** désormais fixé :

- **QDQ** — paires `QuantizeLinear/DequantizeLinear` explicites dans le graphe, portable : n'importe quel runtime peut les fusionner en noyaux int8 à sa façon.
- **QOperator** — opérateurs quantifiés dédiés directement dans le graphe, plus compact mais moins portable.

Taille du fichier (**u8s8** dans les deux cas) :

Format	Taille
QOperator	2,80 Mo
QDQ	3,12 Mo

QOperator produit un fichier ~10% plus léger. Mais la contrainte du projet est un **plafond** de 5 Mo (section 1.1), déjà largement respecté par les deux formats — ce n'est donc pas la taille qui doit trancher ici, contrairement à la vitesse :

u8s8	QDQ	QOperator
Latence médiane	31,4 ms	39,6 ms
IPS	31,8	25,2

QOperator reste plus lent que QDQ même une fois **u8s8** (à 896px, la combinaison la plus défavorable des quatre, **QOperator+s8s8**, atteint même 383,3 ms/2,6 IPS, loin derrière les trois autres). Ayant déjà le budget de taille largement respecté, autant exploiter la marge de vitesse disponible plutôt que de gagner un peu de poids déjà non contraignant.

1. ONNX Runtime, *Quantize ONNX Models* — <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>, section « Data type selection ».

5.4 Calibration PTQ — premier choix et sa limite

Le premier réglage considéré fut la calibration **MinMax** sur 200 images, avec le format **QDQ+u8s8** retenu en section 5.3 — le plus simple pour une première approche, sans hyperparamètre à régler. Résultat sur `test.json` (896px) :

	mAP	AP50	AP75	AP_small	AP_medium	AP_large
fp32	0,485	0,739	0,509	0,335	0,629	0,684
INT8 (MinMax)	0,4377	0,6878	0,4579	0,2897	0,5892	0,6066
Perte vs fp32	-9,7%	-6,9%	-10,0%	-13,5%	-6,3%	-11,3%

Perte substantielle et inégale selon les métriques — particulièrement marquée sur **AP_small** (-13,5%), cohérent avec l'idée que les petits objets, déjà les plus difficiles, sont aussi les plus sensibles à la perte de précision numérique de la quantification.

Cette première approche s'avère sensible aux valeurs aberrantes de calibration : **MinMax** fixe l'échelle de quantification sur le min/max *observé* des activations, une méthode connue pour ce défaut

N images de calibration	mAP
100	0,4428
200 (retenu)	0,4377
500	0,4308
1000	0,4299

Le mAP se **dégrade** de façon monotone quand le nombre d'images de calibration augmente — résultat contre-intuitif, mais cohérent avec le fonctionnement de **MinMax** : plus le nombre d'images grandit, plus la probabilité de rencontrer une activation extrême augmente, étirant artificiellement la plage de quantification au détriment du cas normal majoritaire. Cette sensibilité aux valeurs aberrantes motive directement la recherche d'une méthode de calibration plus robuste.

5.5 Vers une meilleure calibration

Deux méthodes de calibration à histogramme, **Percentile** et **Entropy**, sont conceptuellement conçues pour être plus robustes aux valeurs aberrantes que le min/max brut : plutôt que de fixer l'échelle sur les valeurs extrêmes observées, elles s'appuient sur la distribution complète des activations. Testées dans les mêmes conditions (QDQ+u8s8, N=200, modèle 896px) :

	mAP	AP50	AP75	AP_small	AP_medium	AP_large
fp32	0,485	0,739	0,509	0,335	0,629	0,684
MinMax	0,4377 (-9,7%)	0,6878 (-6,9%)	0,4579 (-10,0%)	0,2897 (-13,5%)	0,5892 (-6,3%)	0,6066 (-11,3%)
Entropy	0,4549 (-6,2%)	0,7063 (-4,4%)	0,4774 (-6,2%)	0,2989 (-10,8%)	0,6070 (-3,5%)	0,6305 (-7,8%)
Percentile	0,4623 (-4,7%)	0,7165 (-3,0%)	0,4877 (-4,2%)	0,3099 (-7,5%)	0,6109 (-2,9%)	0,6522 (-4,7%)

Au-delà de gagner sur les valeurs absolues, **Percentile** est aussi la méthode qui **perd le moins face au fp32** sur **chacune** des 6 métriques, +5,6% de mAP relatif vs MinMax. **Entropy** bat légèrement MinMax mais reste nettement derrière **Percentile**. Le choix retenu sera donc d'utiliser **Percentile** comme méthode de calibration.

5.6 Résultat de quantification (modèle final, 896px)

Décomposition du fichier QDQ final par type de tenseur :

Dtype	Tenseurs	Poids	%
int8	220	2,4208 Mo	89,93%
float32 (<i>scales</i>)	445	0,1357 Mo	5,04%
int32 (biais quantifiés)	220	0,1348 Mo	5,01%
int64 (constantes graphe)	14	0,0003 Mo	0,01%
uint8 (zero-points activation)	224	0,0002 Mo	0,01%
Total	1123	2,6918 Mo	100%

TABLE 1 – Décomposition exhaustive des 1123 tenseurs statiques du modèle final (896px, QDQ, u8s8) par type de donnée brut.

Sans surprise, les poids de convolution (`int8`) dominent totalement le poids du modèle (89,9%) — c’est le seul poste que la quantification per-channel réduit réellement d’un facteur 4 par rapport au `fp32` d’origine. Les biais, systématiquement quantifiés en `int32` par ONNX Runtime (jamais en dessous de 4 octets/valeur, quel que soit le format d’export choisi), et les métadonnées de quantification (*scales* en `float32`, zero-points en `int8/int32/uint8`) ne représentent ensemble que 10,1% du poids total. Les 1123 tenseurs recensés ici ne couvrent que les données brutes stockées dans le graphe : le fichier `.onnx` final pèse 3,121 Mo sur disque, soit 0,429 Mo (13,7%) de plus — un *overhead* de structure (noms des nœuds, attributs des opérateurs, topologie du graphe) plutôt que des poids au sens propre.

Comparaison `fp32` / INT8 final sur les 6 métriques de détection (`test.json`) :

	mAP	AP50	AP75	AP_small	AP_medium	AP_large
fp32	0,485	0,739	0,509	0,335	0,629	0,684
INT8 final	0,462	0,717	0,488	0,310	0,611	0,652
Perte vs fp32	-4,7%	-3,0%	-4,1%	-7,5%	-2,9%	-4,7%

Précision finale, résolution 896px : mAP `fp32` = 0,485 → mAP INT8 = 0,462, soit une perte de **-4,7%** — un peu plus marquée qu’à 768px (-3,2%, mesuré en cours de projet avec la même toolchain), cohérent avec une résolution d’entrée plus fine qui laisse davantage de place à l’erreur de quantification sur les détails fins, sans remettre en cause le choix de la toolchain elle-même.

Contrainte de taille (≤ 5 Mo) respectée avec marge confortable : 3,12 Mo, soit 38% de marge.

6 Performance et compromis précision/vitesse

6.1 Méthodologie de benchmark

Le banc de mesure (`benchmark.py`) applique le même protocole à chaque candidat : 15 itérations de *warmup* (écartées des statistiques, pour ne pas mesurer les effets de cache/JIT à froid), puis 150 mesures. Sont rapportés la moyenne, la médiane, l’écart-type, le minimum, le maximum, le P95 et le IPS dérivé de la médiane. La mémoire résidente (RSS) est échantillonnée toutes les 10 itérations, avec une précaution méthodologique délibérée : la RSS de référence (*baseline*) est mesurée **avant la construction du modèle ou de la session** d’inférence, pas seulement avant la boucle de mesure — pour capturer le coût réel de chargement du modèle en mémoire, pas seulement la croissance pendant les appels répétés. Contrairement à `quantize.py` et `evaluate_int8.py` (section 5), `benchmark.py` ne limite **pas** l’affinité CPU du processus :

la mesure de performance doit refléter les conditions réelles d'exécution, non un environnement artificiellement contraint.

6.2 Résultats comparés

Comparatif au modèle final (896px), fp32 GPU/CPU et candidat de déploiement INT8 :

Candidat	Latence médiane	IPS
PyTorch fp32 GPU	9,39 ms	106,5
PyTorch fp32 CPU	168,15 ms	5,9
ONNX INT8 QDQ CPU (u8s8, retenu)	37,14 ms	26,9

Le **GPU l'emporte nettement** ici (9,39 ms contre 37,14 ms) — cohérent avec un modèle de cette taille, où le calcul brut pèse davantage face au coût fixe du transfert CPU↔GPU. Le candidat retenu reste néanmoins le CPU INT8 : la cible visée par ce projet est l'inférence embarquée (section 1.1), pas un poste équipé d'un GPU dédié.

IPS et mémoire du candidat de déploiement selon la résolution d'entrée (avec même toolchain de quantification) — ici la résolution a un vrai coût, contrairement à la taille du fichier (section 5.6) :

	416px	512px	896px
Latence médiane (CPU, INT8)	9,99 ms	16,50 ms	37,14 ms
IPS (CPU, INT8)	100,1	60,6	26,9
ΔMém (RSS)	12,1 Mo	21,3 Mo	56,5 Mo

La perte de IPS entre 416px et 896px est substantielle (100,1 → 26,9, -73%) et strictement monotone avec la résolution — directement en tension avec le gain de précision, lui aussi monotone et substantiel (mAP fp32 +27,0% entre les deux mêmes points, section 4.4). La mémoire RSS augmente elle aussi avec la résolution du modèle, car les tenseurs d'activations à mémoriser temporairement sur une passe sont plus grand.

6.3 Arbitrage final

896px est retenu comme résolution de déploiement : meilleure précision des trois générations (mAP INT8 = 0,462, section 5.6), et IPS mesuré (26,9) qui **reste au-dessus** du seuil de ≥ 25 IPS retenu (§1.1) — avec une marge plus resserrée qu'anticipé sur la mesure architecture-only de la section 4.3 (28,1 IPS), cohérent avec la variance déjà observée entre sessions de mesure distinctes sur ce poste de développement (section 7).

7 Bilan de la partie détection

Synthèse en une page du modèle finalement déployé (NanoDet-Plus-m-1.5x, 896px, INT8 QDQ u8s8) :

Indicateur	Valeur
Taille fichier (INT8)	3,12 Mo (contrainte ≤ 5 Mo, 38% de marge)
mAP fp32 (test.json)	0,485
mAP INT8 (test.json)	0,462
Perte due à la quantification	-4,7%
IPS mesuré (CPU, INT8)	26,9 (contrainte ≥ 25 IPS, ~8% de marge)

Les quatre contraintes dures du brief (full-CNN, quantifié INT8, taille ≤ 5 Mo, IPS ≥ 25) sont respectées. La progression sur trois générations (section 4) confirme empiriquement l’hypothèse posée dès la section 2.4 : la résolution d’entrée profite spécifiquement aux petits objets (AP_small +91,4% entre 416px et 896px, de loin la métrique la plus transformée), et la toolchain de quantification développée en interne (section 5) ramène la perte due au passage en INT8 à -4,7% sur le modèle final — un résultat obtenu par une démarche itérative et mesurée (calibration MinMax d’abord, limites identifiées, Percentile ensuite) plutôt que par un choix de configuration pris au hasard.

Limites :

- **Déséquilibre person/car jamais éliminé** — le ratio de précision entre les deux classes se resserre au fil des générations (1,44 $\rightarrow$ 1,23, section 4.4) mais reste structurellement >1 à chaque génération, hérité d’un déséquilibre inhérent à COCO lui-même (section 3.2) que le sur-échantillonnage atténue sans l’annuler.
- **Petits objets toujours le point faible** — malgré le gain le plus spectaculaire des trois générations, AP_small (0,335 en fp32 à 896px) reste nettement en dessous d’AP_medium (0,629) et AP_large (0,684) : la hausse de résolution a réduit l’écart, pas comblé le désavantage structurel des petits objets.
- **Marge IPS qui se resserre avec la précision** — 896px ne conserve que $\sim 8\%$ de marge sur le seuil de 25 IPS retenu (contre 400% à 416px, section 6.2) : un compromis assumé, qui permet de rester au-delà du critère fixé à 25 IPS.
- **Mesures faites sur un poste de développement, pas une cible embarquée réelle** — cohérent avec l’absence de contrainte matérielle dans le brief (section 1.1), mais les IPS et la mémoire rapportés ici resteraient à revalider sur le matériel embarqué final, dont les caractéristiques (CPU, présence ou non d’accélération INT8 dédiée) restent inconnues à ce stade.
- **Un seul split fixe, sans validation croisée** — train/val/test proviennent d’un tirage aléatoire unique (graine fixée, section 3.1) : les chiffres rapportés sont ponctuels, sans intervalle de confiance ni estimation de leur variance à travers plusieurs tirages ou plusieurs seeds d’entraînement.
- **Écart de domaine COCO vs cas d’usage réel, non mesuré** — COCO (section 3, choix motivé par la disponibilité de poids pré-entraînés sur ce même corpus) reste un ensemble de photographies grand public, pas des images de caméras de sécurité fixes. Les métriques rapportées dans ce rapport mesurent donc la performance sur la distribution COCO, pas directement sur le cas d’usage visé (section 1) — un écart de domaine plausible, jamais quantifié empiriquement faute de données réelles de ce type disponibles pour ce projet.

8 Suivi multi-objets et journalisation des événements

8.1 Choix de la librairie et architecture générale

Le suivi (*tracking*) s’appuie sur ByteTrackTracker du paquet `trackers` (successeur de `supervision.ByteTrack`, déprécié depuis la version 0.28.0). Le rôle du tracker est de transformer des détections indépendantes d’un frame à l’autre en **pistes** porteuses d’un identifiant stable (`tracker_id`) : condition nécessaire pour répondre à des requêtes comme « depuis combien de temps cette personne est-elle présente », qui exigent de savoir que la boîte vue au frame t et celle vue au frame $t + 1$ désignent le même objet.

La librairie n’a **aucune notion de classe** en interne — une seule instance partagée entre `person` et `car` associerait une détection `car` à une piste `person` par simple recouvrement géométrique (un piéton qui monte dans un véhicule, par exemple). `MultiClassByteTracker` (`agent/src/tracking/tracker.py`) enveloppe donc **une instance de ByteTrackTracker par**

classe, chacune allouant ses propres identifiants à partir de 0 — un même entier `tracker_id` peut donc désigner deux pistes différentes selon la classe, d'où l'usage systématique de la paire (`classe`, `tracker_id`) comme clé partout où l'état est indexé (lissage, hystérésis, journal d'événements).

Le contrat d'entrée/sortie reste celui déjà utilisé dans tout le projet depuis la sortie de `NanoDetPlusHead.post_process` (`{class_idx: [[x1, y1, x2, y2, score], ...], ...}`), avec deux champs ajoutés à chaque boîte en sortie : `tracker_id` et `is_coasted` (voir 8.3).

8.2 Paramètres retenus

Les seuils par défaut de la librairie sont pensés pour des détecteurs à haute confiance ; notre modèle (mAP $\sim 0,46$ en INT8 à 896px, section 5.6) produit des scores plus modestes sur une détection pourtant nette — les seuils ont été abaissés en conséquence, ajustés empiriquement sur des flux vidéos variés, hors dataset COCO :

Paramètre	Retenu	Justification
<code>track_activation_threshold</code>	0,65	Seuil (strict) pour créer une nouvelle piste — abaissé pour un modèle aux scores plus modestes que ceux visés par défaut
<code>high_conf_det_threshold</code>	0,5	Seuil (permissif) qui route une détection vers l'association de 1 ^{re} passe plutôt que la 2 ^e (section 8.3) — distinct du précédent, abaissé pour la même raison
<code>lost_track_buffer</code>	90	Exprimé par la librairie en « frames à 30 FPS » mais utilisé en interne comme une tolérance en secondes d'horloge réelle (<code>maximum_time_without_update = lost_track_buffer / 30.0</code>) — voir discussion ci-dessous
<code>minimum_consecutive_frames</code>	15	Nombre de mises à jour réussies consécutives avant qu'une piste reçoive un <code>tracker_id</code> définitif — relevé pour filtrer les créations de piste sur un faux positif isolé
<code>minimum_iou_threshold</code>	0,1 (inchangé)	Seuil d'IoU minimal pour qu'une association détection $\leftrightarrow$ piste soit retenue par l'algorithme d'assignation (8.3)
<code>frame_rate</code>	30,0 (inchangé)	Voir mise en garde ci-dessous

Le choix de 90 pour `lost_track_buffer` (plutôt que le défaut 30, soit 1 seconde) répond à un besoin concret mesuré empiriquement : avec un affichage graphique réel (`imshow`, redimensionnement 4K $\rightarrow$ 1280px), le pipeline tourne nettement en dessous de 30 FPS — une tolérance d'1 seconde ne représenterait alors que 10 à 15 frames de tolérance réelle, une fenêtre trop courte pour être perceptible à l'oeil lors d'une brève occlusion. Vérifié sur un test dédié : 62 boîtes « en roue libre » sur 120 frames sans délai de rendu simulé, seulement 20 avec un délai de 20 ms/frame représentatif d'un vrai rendu GUI — 90 (3 secondes réelles) redonne une fenêtre de tolérance visible (89 sur le même test) sans faire « fantômer » une piste disparue pendant un temps déraisonnable.

8.3 Logique de traitement, frame par frame

8.3.1 Association détection $\leftrightarrow$ piste (dans `ByteTrackTracker`)

À chaque appel de `update()`, pour chaque classe indépendamment :

1. **Prédiction** — l'état (position, vitesse) de chaque piste existante est avancé d'un pas par son filtre de Kalman interne, calibré sur le temps réellement écoulé depuis la dernière mise à jour (pas un pas fixe par frame).
2. **Élagage préliminaire** — les pistes ayant dépassé leur budget de temps sans association sont retirées avant l'association, pour ne pas polluer l'étape suivante avec des candidats déjà morts.
3. **Séparation des détections par confiance** — les détections du frame sont scindées en deux lots selon `high_conf_det_threshold` : confiance haute et confiance basse.
4. **Association, 1^{re} passe (confiance haute)** — toutes les pistes existantes (prédites) sont mises en correspondance avec les détections de confiance haute par un algorithme d'assignation optimal (`scipy.optimize.linear_sum_assignment`, maximisation de l'IoU — une version sans initialisation de l'algorithme hongrois), une association n'étant retenue que si son IoU dépasse `minimum_iou_threshold`.
5. **Association, 2^e passe (confiance basse)** — les pistes **encore non associées** après la 1^{re} passe tentent une seconde association, cette fois avec les détections de confiance basse — c'est le principe central de ByteTrack : une détection de faible confiance peut malgré tout confirmer une piste déjà établie (typiquement une occlusion partielle), sans jamais pouvoir en créer une nouvelle.
6. **Confirmation de piste** — une piste associée (l'une ou l'autre passe) ne reçoit un `tracker_id` définitif que si elle cumule au moins `minimum_consecutive_frames` mises à jour réussies **consécutives** ; avant ce seuil, elle existe en interne mais ressort avec `tracker_id=-1`.
7. **Création de nouvelles pistes** — seules les détections de confiance haute encore non associées après la 1^{re} passe **et** dont la confiance dépasse `track_activation_threshold` (plus strict que `high_conf_det_threshold`) donnent naissance à une nouvelle piste. Les détections de confiance basse non associées ne créent jamais de piste.
8. **Élagage final** — toute piste à la fois immature (pas encore confirmée) et non associée ce frame-ci, ou ayant dépassé son budget de temps sans association (`lost_track_buffer`), est retirée de la liste.

Point notable : `track_activation_threshold` et `high_conf_det_threshold` jouent des rôles distincts bien que proches en valeur (0,65 et 0,5 retenus) — le premier gouverne uniquement la **naissance** d'une piste, le second gouverne uniquement quelles détections tentent la 1^{re} passe d'association. Une confusion entre les deux mènerait soit à des pistes trop faciles à créer (bruit), soit à des pistes établies perdues trop facilement sur une simple occlusion partielle.

8.3.2 Post-traitement (dans `MultiClassByteTracker`)

Quatre mécanismes supplémentaires sont appliqués **après** le tracker de base, par piste confirmée — tous centralisés dans `config/tracker.yaml` (comme `tracker_kwargs` ci-dessus), sans équivalent dans la librairie `trackers` elle-même :

- **Lissage** — moyenne mobile exponentielle ($\alpha = \text{smoothing_alpha} = 0,55$) sur les coordonnées de boîte affichées, indexée par (`classe`, `tracker_id`) : réduit la gigue visuelle d'une détection à l'autre sans introduire un retard excessif sur le mouvement réel. Relevé par rapport à l'intention nominale (0,4 « à 30 FPS ») pour le même motif que `lost_track_buffer` ci-dessus : au débit de rendu réel mesuré (~ 14 FPS avec un affichage complet), garder 0,4 aurait silencieusement rendu le lissage deux fois plus lent que prévu.
- **Hystérésis d'affichage** (trigger de Schmitt) — une piste doit dépasser `hysteresis_high = 0,4` pour devenir visible, mais ne redevient invisible qu'en repassant sous `hysteresis_low = 0,2` (pas simplement sous `hysteresis_high` à nouveau) — évite qu'un score oscillant juste autour d'un seuil unique fasse clignoter la boîte d'un frame à l'autre. Un score déjà filtré en

amont (ex. seuil de score des scripts de démo) devient ainsi redondant pour tout ce qui sort d'ici : c'est déjà une boîte destinée à être affichée.

- **Extrapolation (« coasting »)** — une piste confirmée, non ré-associée à une détection ce frame-ci mais toujours dans sa fenêtre de tolérance interne, est quand même renvoyée avec `is_coasted=True` et une position lue directement sur l'état du filtre de Kalman (`track.get_state_bbox()`) — un accès à l'état interne de la librairie, non exposé par l'API publique `.update()`, seul moyen d'obtenir cette extrapolation. Réservé aux pistes ayant déjà franchi l'hystérésis pendant qu'elles étaient confirmées : une piste jamais devenue visible ne doit pas se mettre à « fantômer » sans être jamais apparue.
- **Budget de coasting cumulé** — `max_cumulative_predicted_seconds=3,0s` : contrairement à `lost_track_buffer` (section 8.2), qui plafonne chaque *épisode individuel* de coasting, ce budget plafonne le **cumul** sur toute la durée de vie d'une piste. Sans lui, une piste chroniquement à la limite (détections intermittentes qui la ramènent juste avant expiration, sans jamais la confirmer franchement) pourrait fantômer indéfiniment, épisode après épisode. Remis à zéro uniquement quand la piste repasse par une confirmation solide (score $\geq$ `hysteresis_high`) — même ordre de grandeur que `lost_track_buffer` par cohérence d'intention, appliqué ici au cumul plutôt qu'à un seul épisode.

Invariant central : `is_coasted` distingue une boîte réellement associée à une détection ce frame-ci (`False`) d'une boîte en roue libre (`True`). Cette distinction sert **uniquement** à l'affichage (un contour pointillé plutôt que plein dans les scripts de démo, `draw_dashed_rect`) — une extrapolation ne doit **jamais** être transmise au journal d'événements (9.4) comme une observation réelle, sous peine de faire « tourner le temps » d'une piste sur une simple prédiction de mouvement. Cet invariant est appliqué systématiquement à l'appel du journal via une fonction `confirmed_only()` qui écarte les boîtes `is_coasted` avant tout appel à `EventStore.update()` — vérifié présent dans les scripts de démonstration qui combinent tracking et journalisation.

Enfin, à chaque frame, l'état de lissage et d'hystérésis des pistes définitivement abandonnées par le tracker (au-delà de `lost_track_buffer`) est purgé — sans quoi une démonstration tournant longtemps accumulerait indéfiniment de l'état mort en mémoire.

8.4 Journalisation des événements (EventStore)

8.4.1 Schéma et choix de conception

Le journal (`agent/src/journal/event_store.py`) est une base **SQLite** avec trois tables :

Table	Colonnes	Rôle
<code>events</code>	<code>track_id</code> , <code>class</code> , <code>first_seen</code> , <code>last_seen</code>	Une ligne par piste vue au moins une fois
<code>alerts</code>	<code>alert_type</code> , <code>object_class</code> , <code>zone_name</code> , <code>threshold_seconds</code> , <code>person_threshold</code> , <code>car_threshold</code> , <code>count_threshold</code> , <code>window_minutes</code> , <code>created_at</code>	Une ligne par règle configurée, 4 types hétérogènes (<code>duration/co_occurrence/surge/zone</code> , section 9.5) dans une table à colonnes creuses — chaque type ne renseigne que les colonnes qui le concernent
<code>zone_events</code>	<code>id</code> , <code>zone_name</code> , <code>class</code> , <code>track_id</code> , <code>entered_at</code>	Une ligne par entrée en zone (transition dehors→dedans détectée par <code>ZoneMonitor</code> , <code>agent/src/alerts/zones.py</code>), pas une ligne par frame passé dedans

`alerts` utilise une clé composite (`alert_type`, `object_class`, `zone_name`), avec chaîne vide '' (jamais `NULL`) comme sentinelle pour les colonnes non pertinentes à un type donné — SQLite ne fait jamais collisionner deux `NULL` dans une contrainte `UNIQUE`, donc des `NULL` en sentinelle auraient laissé s'accumuler plusieurs règles du même type au lieu de la remplacer (`ON CONFLICT` ne se déclenchant jamais entre deux `NULL`).

La clé primaire de `events` est le couple (`class`, `track_id`) — cohérent avec l'espace de `tracker_id` propre à chaque classe déjà signalé en 9.1. Chaque frame où une piste est vue déclenche un *upsert* (`INSERT ... ON CONFLICT(class, track_id) DO UPDATE SET last_seen = excluded.last_seen`) : la ligne est créée à la première apparition, puis seule `last_seen` est rafraîchie ensuite.

Choix de conception notable : il n'existe **aucun champ « actif »** stocké. Une piste est considérée active si `last_seen` est récent (paramètre `active_within_seconds`), pas via un état à mettre à jour explicitement. Ce choix évite d'avoir à décider frame par frame si une piste doit être marquée « terminée » — décision déjà prise, en pratique, par le tracker lui-même (section 8.3) : une piste peut réapparaître avec le même `tracker_id` après une brève absence tant qu'elle reste dans la fenêtre de tolérance de `lost_track_buffer`, ce que la seule récence de `last_seen` reflète directement, sans double comptabilité à synchroniser entre le tracker et le journal.

8.4.2 Accès concurrent

Le journal est accédé depuis deux threads distincts dans le système complet : la boucle vidéo (tracking, écriture) d'un côté, le thread de l'agent conversationnel (lecture, en réponse à une requête utilisateur) de l'autre. La connexion SQLite est ouverte avec `check_same_thread=False`, associée à un `threading.Lock` explicite posé autour de **chaque** accès à la connexion — la sérialisation des lectures/écritures est ainsi assurée nous-mêmes plutôt que de compter sur un comportement implicite du module `sqlite3`. Point d'implémentation à noter : `active_durations()` délègue entièrement à `active_tracks()` (qui verrouille déjà) **sans** reprendre le verrou lui-même — un `threading.Lock` standard n'étant pas réentrant, un second verrouillage depuis le même thread y provoquerait un interblocage.

8.4.3 Requêtes exposées

Huit méthodes publiques, chacune reprise telle quelle par un tool D1 (section 9.5) — la table `events` seule pour les six premières, `zone_events` pour les deux dernières :

- `count_between(classe, début, fin)` — nombre de pistes distinctes de cette classe dont la **première** apparition tombe dans l'intervalle. Répond à « combien de voitures sont passées entre 12h15 et 12h30 » (`count_between`), et sert aussi de base à `count_total` (borne de fin = maintenant).
- `active_tracks(classe, active_within_seconds)` — pistes vues il y a moins de `active_within_seconds`. Répond à « combien de véhicules sont dans le champ maintenant » (`count_now`).
- `active_durations(classe, active_within_seconds)` — pour chaque piste active, la durée écoulée depuis sa **première** apparition. Répond à « personne présente depuis plus de 10 secondes » (consommé par `AlertMonitor`, section 9.9, pas directement par un tool).
- `most_recent_first_seen(classe)` — timestamp de la piste la plus récemment apparue. Répond à « quand la dernière voiture est-elle passée » (`time_since_last_seen`).
- `average_presence_duration(classe)` — durée moyenne (`last_seen` – `first_seen`) sur toutes les pistes de cette classe depuis le début de la session. Répond à « en moyenne combien de temps reste une personne » (`average_presence_duration`).
- `peak_concurrent_count(classe)` — nombre maximal de pistes actives **simultanément** à un instant quelconque depuis le début de la session. Répond à « quel a été le pic de véhicules » (`peak_concurrent_count`).
- `log_zone_entry(zone, classe, track_id)` — écrit une ligne dans `zone_events` ; appelée par `ZoneMonitor` à chaque entrée détectée, jamais directement par un tool.
- `count_zone_entries(zone, classe)` — nombre d'entrées journalisées dans cette zone. Répond à « combien de fois une voiture est entrée dans le quai de chargement » (`count_zone_entries`).

Les méthodes basées sur une fraîcheur (`active_tracks`, `active_durations`) utilisent volontairement une tolérance **courte** par défaut (1 seconde). Cette tolérance ne compense **aucune latence d'exécution** (voir section 9.7 pour la démonstration) : elle absorbe uniquement le clignotement occasionnel du détecteur (quelques frames ratées sur un objet pourtant toujours présent). C'est une question produite à part entière, pas un simple délai technique à deviner.

9 Agent conversationnel

L'agent répond en langage naturel aux questions portant sur les détections journalisées en section 8 — compter les objets présents, filtrer par plage horaire, configurer une alerte de présence continue, etc... (cahier des charges, section 1.1). Deux composants : les **tools** qui font le pont entre une question et une requête sur `EventStore` (`agent/src/agent/tools.py`), et la **boucle agent** qui orchestre l'appel au LLM et l'exécution des tools qu'il demande (`agent/src/agent/agent.py`).

9.1 Choix du runtime et critères de sélection

Le runtime retenu est `llama.cpp` directement, plutôt qu'Ollama — qui l'utilise de toute façon en interne pour un usage mono-utilisateur local, la différence sous le capot est marginale. Ce choix est cohérent avec la philosophie déjà suivie côté détection (`onnxruntime` plutôt que PyTorch complet en déploiement, section 5) — un moteur d'inférence C++ léger, sans dépendance à un framework d'entraînement, pour le LLM comme pour la détection.

Le choix du modèle lui-même répond à des critères explicites, dans le même esprit méthodologique que la sélection d'architecture de détection (section 2.2) :

- **Handler de tool-calling natif dans `llama.cpp`** — un parseur dédié au format d'appel de tools du modèle, plus fiable que le *fallback* générique². Critère éliminatoire, comme le « graphe exporté propre » l'était pour l'architecture de détection.
- **Licence compatible avec un usage en entreprise** — même raisonnement que l'exclusion de YOLOv8n en AGPL-3.0 (section 2.2) : un modèle sous licence non-commerciale est écarté quel que soit son mérite technique.
- **Budget VRAM disponible** — 8 Go sur la RTX 4060 déjà utilisée pour l'entraînement (section 4), à ne pas saturer.
- **Validation empirique par un banc de test dédié** plutôt que par un chiffre proxy : contrairement au mAP COCO pour la détection (section 2.4), il n'existe pas de benchmark public directement transférable à *notre* jeu de 12 tools — la seule mesure fiable est un banc de test construit sur mesure (section 9.3).

9.2 Comparatif étendu des modèles candidats

2. Liste officielle des formats de tool-calling supportés : `docs/function-calling.md`, dépôt `ggml-org/llama.cpp`.

Modèle	Params	Taille Q4_K_M	VRAM totale estimée*	Marge/8 Go	Licence	Remarque
Qwen2.5-7B-Instruct	7B	4,68 Go (confirmé)	~5,5–6 Go	confortable	Apache 2.0	Référence connue, déjà validée de bout en bout avant ce comparatif étendu
Hermes-3-Llama-3.1-8B (Nous Research)	8B	4,92 Go (confirmé)	~5,7–6,2 Go	confortable	Communautaire Llama 3.1 (héritée)	Format de tool-call (Hermes) réputé fiable en JSON structuré
Functionary-small-v3.2 (MeetKai)	8B	4,92 Go (bartowski) / 4,66 Go en Q4_0 (dépôt officiel, seul quant. dispo)	~5,7–6,2 Go	confortable	Communautaire Llama 3.1 (héritée, base Llama-3.1-8B)	Seul modèle conçu exclusivement pour le tool-calling, mais support des appels parallèles cassé côté <code>llama.cpp</code> (section 9.3.3)
Command R7B (Cohere)	~8B	5,06 Go (confirmé)	~5,9–6,4 Go	confortable	CC-BY-NC (non-commercial)	Écarté sans test — même raisonnement que l'exclusion AGPL de YOLOv8n (section 2.2)
Granite-4.1-3B (IBM)	3B	2,10 Go (confirmé)	~2,8–3,3 Go	très large	Apache 2.0	Beaucoup plus rapide, marge VRAM énorme — capacité de raisonnement sur l'extraction de paramètres fins à valider empiriquement
Mistral-Nemo-Instruct-2407	12B	7,48 Go (confirmé)	~8,3–8,8 Go	dépasse 8 Go	Apache 2.0	Écarté sans test — seul candidat dépassant le budget VRAM mesuré

* *VRAM totale = poids + KV-cache/overhead runtime, estimée selon le même ratio que la mesure réelle de Qwen2.5-7B (4,68→~5,5–6 Go) — à confirmer par mesure directe pour chaque nouveau candidat, pas une valeur garantie a priori.*

Sur les 6 candidats, 2 sont écartés sans même être testés : Command R7B sur un critère non technique (licence CC-BY-NC), Mistral-Nemo parce qu’il dépasse seul le budget VRAM mesuré de 8 Go, sans laisser de marge pour le KV-cache. **Les 4 candidats restants sont soumis à un banc de test réel** (section 9.3) — Qwen2.5-7B-Instruct, Hermes-3-Llama-3.1-8B, Functionary-small-v3.2 et Granite-4.1-3B.

9.3 Banc de test réel

9.3.1 Méthodologie

Une banque de **36 questions** (`cases.py`) couvrant les 12 tools exposés (section 9.5) ainsi que des variantes et pièges délibérés — synonymes, formulation en anglais, désambiguïsation « maintenant » vs « total », horaires absolus et relatifs, appels multiples dans un même tour, zone inconnue, questions hors périmètre. Chaque question est rejouée **10 fois** — la sortie d’un LLM n’est pas déterministe d’un tick à l’autre (sampling). Un cas qui échoue une fois sur dix n’est pas équivalent à un cas qui échoue toujours.

Le prompt système utilisé est **celui de la production** (`config/agent.yaml`, chargé dynamiquement, jamais recopié en dur dans le banc de test) — pour ne jamais valider un prompt différent de celui réellement déployé. Le scoring (`harness.py:score_case`) est **strict** : appariement glouton, indépendant de l’ordre, entre les appels de tools attendus et les appels réellement effectués (nom + arguments) ; un cas n’est compté « réussi » que si **tous** les appels attendus sont retrouvés **et** qu’**aucun** appel superflu ne reste — un tool de trop (ex. compter les deux classes quand une seule était demandée) est une vraie erreur de comportement. .

9.3.2 Résultats agrégés

Modèle	Taux de réussite	Latence médiane	Latence P95	VRAM confirmée
Granite-4.1-3B	97,2%	0,65 s	1,01 s	2,10 Go
Hermes-3-Llama-3.1-8B	93,9%	1,13 s	1,58 s	4,92 Go
Qwen2.5-7B-Instruct	92,8%	1,24 s	2,03 s	4,68 Go
Functionary-small-v3.2	90,6%	1,03 s	1,94 s	4,66 Go

Chiffres finaux, 10 répétitions par cas (360 exécutions par modèle).

Granite-4.1-3B se distingue nettement sur les trois axes mesurés : meilleur taux de réussite (97,2%), latence médiane la plus basse (0,65 s, $\sim 2\times$ plus rapide que les trois autres) et empreinte VRAM la plus faible (2,10 Go). Hermes-3 et Qwen2.5-7B se tiennent proches en précision (93,9% et 92,8%) mais restent nettement plus lents et plus lourds. Functionary-small-v3.2 termine dernier sur les trois métriques — sa latence mesurée est en outre faussée à la baisse par des échecs précoces et fréquents (section 9.3.3), donc probablement plus optimiste que sa latence réelle sur un échange qui aboutit normalement.

9.3.3 Analyse des échecs

Erreurs restantes, propres à chaque modèle, confirmées sur l’échantillon à 10 répétitions :

- **Qwen2.5-7B** : convertit systématiquement « midi » en 13:00/13:15 au lieu de 12:00/12:15 sur 2 cas indépendants (`count_since__loose_natural_phrasing`, 0/10 ; `count_between__morning`, 2/10) — une erreur de compréhension déterministe, pas de la formulation non standard. Sur-tout, sur `multi__compare_now` (2 appels nécessaires), **7/10 runs partent en boucle** (40 à 49 appels `count_now` alternés sans converger vers une réponse), dont **3 font planter le serveur** (JSON d’arguments malformé côté `llama.cpp`) — ce que l’échantillon à 3 répétitions ne laissait voir que comme un cas isolé se révèle être le comportement **majoritaire** de Qwen sur ce cas précis.

- **Hermes-3-8B** : confond systématiquement « depuis 9h du matin » (heure absolue) avec « il y a 9 minutes » (`minutes_ago`: 9, 8/10) — erreur de compréhension plus grave que celle de Qwen sur un cas du même type. Flou sur les formulations temporelles relatives (« cette dernière demi-heure », 6/10; « midi et quart », 7/10, parfois interprété comme une durée relative erronée). Sur `multi__compare_now`, 6/10 runs corrects; les 4 échecs ajoutent un appel `count_now` redondant plutôt que de partir en boucle incontrôlée — une dérive contenue, sans commune mesure avec celle de Qwen sur le même cas.
- **Granite-4.1-3B** : confirmé comme le point faible principal, « cette dernière demi-heure » ne passe que **3/10** — le modèle glisse vers un horaire absolu *inventé* plutôt que d'utiliser `minutes_ago`, avec des valeurs qui varient d'un run à l'autre (10:00, 13:30, 15:00...), signe qu'il n'a pas de référence fiable à « maintenant » pour ce type de formulation. C'est nettement plus fréquent que ne le suggérait l'échantillon à 3 répétitions (2/3). En dehors de ce cas, **aucune autre erreur** sur les 35 cas restants.
- **Functionary-small-v3.2** : le moins fiable des 4 sur cet échantillon élargi. Confond systématiquement une alerte de durée de présence en anglais avec une alerte de seuil (`set_surge_alert` au lieu de `set_presence_duration_alert`, 9/10). Le crash de grammaire déjà documenté sur `multi__compare_now` (section 9.3.1) se confirme **beaucoup plus fréquent** qu'estimé sur 3 répétitions : **9/10 runs plantent le serveur**, pas un cas isolé. Deux cas auparavant propres deviennent légèrement flaky sur l'échantillon élargi (`time_since_last_seen__car` 9/10, `peak_concurrent_count__car` 8/10). **Nuance de mesure** : sur `count_between__morning` (6/10), les échecs viennent en partie d'un horaire rendu "8:00" plutôt que "08:00" attendu par le scoring strict (section 9.3.1) — une réponse sémantiquement correcte mais pénalisée par l'appariement exact, à distinguer d'une vraie erreur de raisonnement.

9.4 Décision finale : modèle retenu

Granite-4.1-3B est retenu. Premièrement la détection tourne exclusivement sur CPU en déploiement (`CPUExecutionProvider`, section 6.2) — elle ne consomme donc pas de VRAM à proprement parler. Mais le script de démo complet (`demo/src/scripts/03_live_agent_demo.py`) fait tourner le pipeline vidéo (capture, tracking, journal, rendu `cv2.imshow`) et le serveur LLM **en parallèle, sur la même machine**, équipée d'un **unique GPU** (RTX 4060, 8 Go) — la seule ressource GPU disponible ce jour-là doit donc absorber non seulement le serveur LLM, mais aussi tout ce qui tourne par ailleurs sur ce même poste pendant une présentation en direct (partage d'écran ou enregistrement, compositeur du bureau, navigateur). Avec Qwen2.5-7B ou Hermes-3-8B (4,68–4,92 Go de poids seuls, ~5,5–6,2 Go une fois le KV-cache compté), la marge restante sur 8 Go est confortable en isolation mais se resserre vite dès que d'autres processus sollicitent le GPU. **Granite-4.1-3B (2,10 Go de poids, ~2,8–3,3 Go au total) libère 2,5 à 3 Go de VRAM supplémentaires** — une marge de sécurité réelle. Ensuite Granite obtient **97,2%**, le **meilleur taux de réussite confirmé des 4 candidats**. Granite reste par ailleurs **~2× plus rapide** que tous les autres (médiane 0,65 s contre 1,03–1,24 s). Ce gain de vitesse compte directement pour ce genre de projet où la latence entre la requête et la réponse de l'agent peut entraîner des conflits sur la fraîcheur des données. La section 9.7 traite de ce risque dès la conception. Son principal point faible confirmé (formulations temporelles relatives floues, 3/10 sur « cette dernière demi-heure », section 9.3.3) reste une erreur de sélection d'argument, pas une erreur de compréhension temporelle grave comme celle d'Hermes sur « 9h du matin » ni une instabilité structurelle comme les boucles incontrôlées de Qwen sur `multi__compare_now` (7/10 runs, dont 3 crashes serveur) — un type d'erreur plus inquiétant pour un usage produit réel. Licence Apache 2.0, et aucun patch de template nécessaire (contrairement à Hermes/Functionary).

9.5 Les tools

12 tools, tous généralisés par `object_class` ("person" ou "car") plutôt que dupliques par classe, regroupés en 3 catégories (`agent/src/agent/tool_schemas.py`) :

- **Comptage** — `count_now` (présents actuellement), `count_total` (distincts depuis le début de la session), `count_since` (depuis un point de départ, heure absolue *ou* durée relative), `count_between` (entre deux horaires précis).
- **Statistiques dérivées** — `time_since_last_seen` (délai depuis la dernière apparition), `average_presence_duration` (durée moyenne de présence), `peak_concurrent_count` (pic d'objets simultanés).
- **Alertes (configuration uniquement)** — `set_presence_duration_alert` (présence continue trop longue), `set_co_occurrence_alert` (seuils combinés personnes ET véhicules), `set_surge_alert` (afflux/rafale en fenêtre glissante), `set_zone_alert` et `count_zone_entries` (zones de danger prédéfinies). Ces tools n'exécutent pas eux-mêmes la surveillance de la condition, rôle du mécanisme d'alerte (`agent/src/alerts/alerts.py`).

9.6 Boucle agent et function-calling

`Agent.ask()` envoie la question au serveur (`chat.completions.create`) avec un prompt système en français et la liste des tools disponibles. Deux issues possibles à chaque tour : soit le serveur renvoie directement une réponse textuelle (fin de boucle), soit il renvoie un ou plusieurs `tool_calls`. Dans ce second cas, chaque appel passe par une table de dispatch (nom de tool → méthode Python réelle), volontairement protégée : toute exception levée par un tool est capturée et transformée en message d'erreur lisible plutôt que de remonter jusqu'à l'appelant — pour que le LLM puisse **réagir** à un échec (reformuler, réessayer avec d'autres paramètres) plutôt que de faire planter toute la conversation. Le résultat de chaque tool est réinjecté dans l'historique des messages, et une nouvelle requête est envoyée au serveur — jusqu'à obtenir une réponse textuelle finale, ou épuiser un plafond de 4 tours (`max_rounds`).

9.7 Latence LLM et fraîcheur des données

Point anticipé dès la conception : un tool d'agent ne s'exécute jamais au moment exact où la question est posée, mais **après** le temps de décision du LLM. Or lorsque le tool interroge la base de donnée, il doit le faire relativement à l'instant de la requête et non pas à l'instant $t + \text{latence LLM}$ afin de pouvoir remonter un résultat cohérent avec l'état réel du système à l'instant de la requête utilisateur. On capture donc `request_ts = time.time()` dans `Agent.ask()` **avant** tout appel au LLM, puis on l'injecte explicitement (paramètre `now`, jamais exposé au LLM — absent de `tool_schemas.py`) au moment d'exécuter le tool. « Maintenant » signifie alors « au moment où la question a été posée », plus jamais « au moment où le tool a fini par s'exécuter » — le résultat devient **indépendant de la latence d'inférence**, quelle qu'elle soit, plutôt que de dépendre d'un seuil de tolérance élargi de façon arbitraire.

9.8 Mémoire conversationnelle (D2)

Chaque appel à `Agent.ask()` reconstruit une liste de messages **neuve**, sans aucun souvenir de l'échange précédent. Une question elliptique comme « et de voitures ? » juste après « combien de personnes actuellement ? » n'a alors aucun moyen d'être comprise. L'implémentation d'une mémoire basique vise à régler ce soucis.

9.8.1 Risques identifiés avant implémentation

Trois risques posés dès la conception, avant d'écrire la moindre ligne de code, plutôt que découverts après coup :

- **Concurrence avec la gestion des alertes** — `Agent.ask()` est désormais appelé depuis deux threads (`input_worker` pour les questions utilisateur, `alert_notifier` pour les notifications d’alerte, section 9.9). Un historique **partagé** entre les deux exposerait une vraie condition de course (deux threads mutant la même liste en même temps).
- **Réutilisation de valeurs périmées** — rien ne garantit qu’un modèle, voyant un chiffre déjà donné dans l’historique, rappelle le tool pour une nouvelle question sur l’état *actuel* plutôt que de recycler ce chiffre obsolète.
- **Croissance du contexte** — `llama-server` tourne avec `-c 8192` tokens ; un historique qui s’accumule sans borne finirait par déborder sur une session longue.

9.8.2 Design retenu

- **Session, portée par l’appelant, jamais par Agent** — chaque appelant garde sa propre instance (`input_worker` en garde une pour toute la durée du script ; `alert_notifier` n’en crée volontairement **aucune**, chaque notification restant un échange isolé). Règle le risque de concurrence par construction : deux threads ne se retrouvent jamais à muter la même liste.
- **Consigne anti-péremption dans le prompt système** — ajout explicite : un chiffre obtenu par un tool devient obsolète dès qu’il est donné, le tool correspondant doit être rappelé pour toute nouvelle question sur l’état actuel, même déjà posée auparavant.
- **Fenêtre glissante** (`memory.max_turns`, 6 par défaut) — coupe uniquement à une frontière d’échange complet, jamais en plein milieu d’une séquence de tool calls (un historique tronqué y laisserait des messages tool sans le `tool_call` correspondant, rejeté par l’API).
- **Rollback sur échec** — une question qui échoue (erreur réseau) ou qui abandonne (`max_rounds` atteint) ramène l’historique exactement à son état d’avant cette question : un échange sans réponse ne doit jamais polluer les tours suivants.

9.8.3 Extension du banc de test et résultat mesuré

Le banc de test existant (section 9.3) est strictement mono-tour — étendu par **5 séquences multi-tours** dédiées (`agent/eval/cases.py:MULTI_TURN_CASES`, script séparé `run_memory_benchmark.py`), sans toucher une seule ligne du banc mono-tour existant (36 cas, `run_benchmark.py`) : chaque séquence partage un historique d’un tour à l’autre via `RecordingAgent.ask_continuing`, une méthode neuve plutôt qu’une modification de `ask()` pour garder zéro risque de régression sur les cas déjà validés.

Séquence	Tours	Ce qu’elle valide
<code>followup_class_swap</code>	2	Cas de référence : « et de voitures ? » compris à la lumière du tour précédent
<code>followup_alt_phrasing</code>	2	Formulation elliptique différente (« côté X » plutôt que « et de X ? »), sur un autre tool
<code>no_stale_reuse</code>	2	La même question sur « maintenant » redéclenche le tool, ne réutilise pas le chiffre déjà donné (risque n°2 ci-dessus)
<code>unrelated_topic_no_confusion</code>	2	Un changement de sujet complet ne fait pas dériver le tour suivant vers l’ancien sujet
<code>three_turn_chain</code>	3	Suivi de contexte robuste au-delà d’un aller-retour unique

Résultat sur Granite-4.1-3B (modèle retenu, section 9.4), serveur réel : 5/5 séquences réussies intégralement, chaque tour correct. En particulier `no_stale_reuse` confirme que la consigne ajoutée au prompt système est effectivement suivie, pas seulement présente dans le texte.

9.9 Alertes en temps réel : conception silencieuse et notification par l’agent

(`demo/src/scripts/03_live_agent_demo.py`) surveille les règles d’alerte configurées par l’agent sans jamais perturber la console qui sert à taper les questions :

- **Surveillance silencieuse** — `AlertMonitor/ZoneMonitor` tournent à chaque frame ; toute leur activité est tracée dans un fichier log dédié (`logging`), jamais sur la console.
- **Bannière immédiate** en haut de la fenêtre dès qu’une alerte se déclenche — retour visuel instantané, sans attendre le LLM.
- **Notification par l’agent, en arrière-plan** — `alert_notifier`, un thread daemon de courte durée par alerte (même principe qu’`input_worker`, jamais dans la boucle vidéo), transforme l’événement brut en phrase naturelle via un tour de LLM — l’appel réseau est protégé par un `try/except` (même principe qu’en section 9.6), pour qu’un serveur LLM injoignable ne fasse jamais planter le thread. Le résultat est déposé dans la **même file** que les réponses aux questions tapées — la boucle vidéo n’a pas besoin de savoir d’où vient une réponse pour l’afficher.
- **Aucune alerte pré-configurée par défaut** — c’est à l’agent de poser une règle d’alerte, uniquement si l’utilisateur le demande.
- **État remis à zéro au démarrage** — `store.clear_events()` est appelé systématiquement au lancement du script, en plus du changement de source : les `tracker_id` repartant toujours de 0, une nouvelle piste `#0` ne doit jamais hériter du `first_seen` d’une ligne laissée en base par une session précédente.

10 Intégration système et démonstration live

Cette section décrit l’architecture d’intégration et les résultats mesurés sur la démonstration complète — les instructions d’installation, de configuration et de lancement du projet (y compris de ce script de démonstration) font l’objet d’un document séparé (`README.md` du dépôt), pour ne pas mélanger analyse et guide opérationnel.

10.1 Architecture du pipeline complet

`demo/src/scripts/03_live_agent_demo.py` est le point d’entrée unique qui assemble, en un seul processus, toutes les briques développées séparément dans les sections précédentes : détection (ONNX INT8, section 5–6), tracking multi-classe (section 8.1–8.3), journalisation des événements (`EventStore`, section 8.4) et agent conversationnel avec alertes en temps réel (section 9).

Quatre threads se partagent le travail :

- **frame_producer** — capture et prétraitement (warp/resize + normalisation + conversion en tenseur) de la frame suivante, en continu, pendant que le thread principal traite encore la précédente (voir section 10.2 pour la justification de ce découpage).
- **Boucle vidéo (thread principal)** — inférence (détection), tracking, journalisation des boîtes confirmées (`confirmed_only()`, section 8.3) et rendu `cv2.imshow`, en continu.
- **input_worker** — lit les questions tapées par l’utilisateur (`input()`) et les transmet à `Agent.ask()` (section 9.6), en conservant sa propre `Session` pour la mémoire conversationnelle (section 9.8).
- **alert_notifier** — un thread daemon de courte durée par alerte déclenchée, qui transforme l’événement brut en phrase naturelle via un tour de LLM (section 9.9).

Ces quatre threads ne communiquent **jamais directement** entre eux : ils passent par des points de synchronisation dédiés, chacun protégé par son propre verrou ou sa propre file —

`EventStore` (`threading.Lock`, section 8.4) pour l'écriture/lecture des événements, une file de sortie commune (section 9.9) pour tout ce qui doit s'afficher à l'utilisateur (réponse à une question ou notification d'alerte), et une file dédiée (taille 2) entre `frame_producer` et le thread principal pour les frames prétraitées — `cycler` (section source, changement de vidéo) n'est manipulé que par `frame_producer`, jamais par le thread principal, pour éviter une double mutation concurrente.

La détection tourne exclusivement sur CPU (`CPUExecutionProvider`, section 6.2), tandis que le serveur LLM (`llama-server`) occupe seul le GPU (RTX 4060, 8 Go). Côté CPU (12 cœurs logiques), trois bibliothèques se partagent désormais explicitement les cœurs entre elles — `onnxruntime` (inférence, thread principal), `OpenCV` (prétraitement, `frame_producer`) et `PyTorch` (post-traitement/NMS et conversion tenseur, les deux threads) — voir section 10.2 pour la mesure qui a motivé ce partage.

10.2 Performance de bout en bout

10.2.1 Diagnostic : où part le temps

Le **FPS réel** de la démonstration complète (capture, prétraitement, inférence, post-traitement, tracking, journal, rendu) était initialement de **~14 FPS** — nettement inférieur à l'IPS du modèle seul (26,9 IPS, section 6.2). Cet écart est attendu et volontairement distingué dès l'introduction (section 1.1) : l'IPS mesure la fréquence d'inférence pure du modèle, hors tout le reste du pipeline. Un profilage étape par étape (méthodologie `benchmark.py` — warmup 15 + 150 mesures — sur `autumn_street_car.mp4`, 1920×1080, poste isolé de toute autre charge CPU) a permis de quantifier précisément cet écart :

Étape	Médiane	% du total
Décodage vidéo	3,6 ms	4,6%
Prétraitement (warp+resize+normalize)	19,4 ms	24,5%
Conversion tenseur	4,9 ms	6,2%
Inférence ONNX	35,6 ms	44,9%
Post-traitement (boîtes/NMS)	7,9 ms	9,9%
Tracking	0,7 ms	0,8%
Journal SQLite	4,3 ms	5,5%
Alertes	0,2 ms	0,2%
Dessin overlay	0,4 ms	0,5%
Resize affichage	0,8 ms	1,0%
<code>imshow+waitKey</code>	1,6 ms	2,0%
Total	≈79,2 ms	100%

On y voit que le prétraitement + conversion tenseur + post-traitement totalisent **32,1 ms (40,6% du total)** — presque autant que l'inférence elle-même. Ni le décodage vidéo ni l'affichage ne sont en cause (respectivement 4,6% et 2,0% à eux deux) : le vrai poste optimisable, au-delà de l'inférence, est ce trio prétraitement/ conversion/post-traitement, exécuté **séquentiellement** avant cette optimisation.

10.2.2 Recouvrir prétraitement et inférence

Puisque le prétraitement d'une frame ne dépend pas du résultat d'inférence de la précédente, les deux peuvent en principe se chevaucher plutôt que s'additionner. Un thread `frame_producer` dédié (section 10.1) a été ajouté à `04_live_agent_demo.py` : il lit et prétraite la frame $N + 1$ pendant que le thread principal infère encore sur la frame N .

Première tentative, sans réglage particulier : un résultat négatif. Le FPS mesuré a reculé (10,7–12,5 FPS, pire que la version séquentielle) plutôt que de progresser, et l'inférence

isolée elle-même a ralenti de 35,6 ms à 56–93 ms. Cause : `OpenCV` (prétraitement), `onnxruntime` (inférence) et `PyTorch` (post-traitement/NMS) sont chacun **déjà multi-threadés en interne**, et essaient chacun de leur côté de saturer les 12 cœurs logiques disponibles pour une seule opération. Faire tourner prétraitement et inférence dans deux threads séparés ne les fait donc pas se partager le travail proprement : les trois bibliothèques se disputent les mêmes cœurs (sursouscription), et l’inférence en pâtit directement.

Deuxième tentative : répartir explicitement les cœurs, sans en donner plus que nécessaire à l’inférence. Une mesure isolée de l’IPS selon le nombre de threads `onnxruntime` alloués (`intra_op_num_threads`) montre des rendements décroissants au-delà de 8–10 threads (29,4–29,8 IPS), voire une légère régression à 12 threads (27,8 IPS) — donner *tous* les cœurs à l’inférence n’est donc déjà pas optimal en soi. Le seuil dur du projet étant ≥ 25 IPS (section 1.1), pas la maximisation de l’IPS, l’objectif est le plus petit budget de cœurs pour `onnxruntime` qui reste au-dessus de ce plancher, le reste étant libéré pour le prétraitement et le post-traitement — une ablation complète de ce partage est détaillée juste après (section 10.2.3), et retient au final **6 threads `onnxruntime` / 2 `OpenCV` / 4 `PyTorch`** (sur 12 cœurs logiques).

	FPS réel	Inférence isolée (IPS)
Séquentiel (avant optimisation)	12,6–13,4	26,9
Pipeliné, réglage par défaut	10,7–12,5 (pire)	10,7–17,8
Pipeliné, cœurs répartis 9/1/2	17,0–17,4	25,8–27,1
Pipeliné, cœurs répartis 6/2/4 (retenu)	18,2	25,5

Un premier partage (9 threads `onnxruntime` / 1 `OpenCV` / 2 `PyTorch`) a d’abord été confirmé en conditions réelles (fenêtre vidéo + webcam, pas seulement en mesure isolée) : **16–17 FPS**, contre ~ 14 FPS avant — déjà un gain net de ~ 20 – 25% . L’ablation détaillée ci-dessous (section 10.2.3) a ensuite montré qu’un partage plus agressif (6 threads `onnxruntime` / 2 `OpenCV` / 4 `PyTorch`) pousse le FPS réel un cran plus loin (18,2, mesuré avec la même méthodologie headless) au prix d’une marge IPS plus fine (25,5, $\sim 2\%$ au-dessus du seuil de 25 plutôt que $\sim 6,4\%$) — **ce partage 6/2/4 est le choix retenu au final**, la marge restant jugée suffisante au regard du gain de FPS.

10.2.3 Ablation : combien de cœurs donner à l’inférence ?

La répartition 6/2/4 retenue ci-dessus n’a pas été choisie au premier essai — elle vient d’une exploration plus large du nombre de cœurs `onnxruntime` (le reste étant à chaque fois réparti entre `OpenCV` et `PyTorch`, meilleur partage retenu à chaque palier), menée pour situer précisément le point de bascule sous le plancher de 25 IPS :

<code>onnxruntime</code>	<code>OpenCV</code>	<code>PyTorch</code>	FPS réel	IPS inférence
12 (aucun partage)	12	12	10,7	17,8 (sous 25)
9	1	2	17,2	26,6
8	2	2	17,1	26,8
6	2	4	18,2	25,5
5	2	5	18,4	24,9 (sous 25)
4	3	5	17,9	23,8 (sous 25)

Le FPS continue de grimper légèrement à mesure qu’on retire des cœurs à `onnxruntime` ($10,7 \rightarrow 18,4$ entre 12 et 5 threads) — logique, puisque cela libère d’autant plus de cœurs pour le recouvrement prétraitement/post-traitement. Le point de bascule est net : l’IPS de l’inférence **passse sous le plancher de 25** dès 5 threads (24,9), et encore plus nettement à 4 (23,8) — ces deux paliers sont donc exclus d’office, contrainte dure du projet oblige (section 1.1), quel que soit le gain de FPS qu’ils offrent par ailleurs. **6 threads `onnxruntime` est le dernier palier**

qui respecte encore le plancher (25,5 IPS, $\sim 2\%$ de marge) — plus fine qu’à 9 threads (26,6 IPS, $\sim 6,4\%$), mais pour +1 FPS réel (18,2 contre 17,2), un compromis jugé préférable : **6/2/4** est donc le partage retenu.

10.3 Scénario validé

La démonstration a été exercée de bout en bout sur les capacités suivantes :

- détection et tracking en continu sur un flux vidéo réel, hors dataset COCO ;
- les 3 requêtes d’exemple du cahier des charges (section 1.1), déjà validées contre le vrai serveur en section 9.7 ;
- configuration d’une alerte de présence continue et d’une alerte de zone à la volée, avec notification effective par l’agent (section 9.9) ;
- résilience à un serveur LLM temporairement injoignable, sans interrompre la boucle vidéo ni faire planter les threads d’arrière-plan (section 9.9).

11 Synthèse générale et perspectives

11.1 Bilan par rapport au cahier des charges

Les deux livrables visés dès l’introduction (section 1.1) sont délivrés et intégrés en une démonstration unique fonctionnelle (section 10) :

- **IA Embarquée** — les quatre contraintes dures (full-CNN, quantifié INT8, ≤ 5 Mo, ≥ 25 IPS) sont respectées par le modèle final (bilan détaillé, section 7).
- **IA Agentique** — les capacités demandées (comptage, filtrage par plage horaire, alertes de présence continue et de zone) sont implémentées et testées, avec un modèle sélectionné sur la base d’un banc de test construit sur mesure plutôt que sur un chiffre proxy (section 9.3–9.5).

11.2 Enseignements transversaux

Au-delà des résultats chiffrés, plusieurs principes se retrouvent de façon répétée dans les choix faits tout au long de ce projet :

- **Mesurer plutôt que supposer** — que ce soit pour choisir une méthode de calibration INT8 (section 5.4–5.5), pour départager des modèles de LLM candidats (section 9.3) ou pour dimensionner les paramètres du tracker sur le débit réel de la démonstration plutôt que sur une valeur nominale (section 10.2), la même démarche revient : construire un banc de test reproductible avant de trancher, plutôt que de se fier à un chiffre publié ou à une intuition.
- **Concevoir pour pouvoir revenir en arrière** — un interrupteur de configuration unique pour la mémoire conversationnelle (`memory.enabled`, section 9.8), un `save_dir` distinct à chaque réentraînement plutôt qu’un écrasement — la réversibilité d’un choix a été traitée comme une propriété à concevoir dès le départ.
- **Anticiper les échecs plutôt que les découvrir en prod** — l’ancrage temporel de `request_ts` (section 9.7) ou la robustesse aux pannes réseau du serveur LLM (section 9.9) ont été traités comme des risques identifiés en amont, avant qu’ils ne se manifestent pendant une démonstration en direct.

11.3 Limites globales

Les limites propres à la partie détection sont détaillées en section 7. À l'échelle du système complet, s'ajoutent :

- l'absence de validation sur une cible matérielle embarquée réelle, aussi bien pour le modèle de détection seul (section 7) que pour l'intégration complète ;
- un point faible confirmé de l'agent sur les formulations temporelles relatives floues (« cette dernière demi-heure », 3/10 sur Granite, section 9.3.3), à surveiller si le périmètre de questions s'élargit ;
- un écart de domaine jamais quantifié entre COCO (entraînement) et des images réelles de caméras de sécurité fixes (section 3, introduction).

11.4 Pistes de prolongement

- **Validation sur cible embarquée réelle** — rejouer les mesures d'IPS, de mémoire et de FPS de bout en bout (sections 6 et 10.2) sur le matériel effectivement visé par un déploiement, une fois celui-ci spécifié.
- **Réduire l'écart de domaine** — collecter et annoter un petit corpus d'images issues de caméras de sécurité réelles, pour quantifier puis réduire l'écart avec COCO (section 3).
- **Quantization-aware training** — explorer un entraînement qui intègre la quantification plutôt qu'une PTQ pure (section 5), pour tenter de réduire davantage la perte de -4,7% observée sur le modèle final (section 5.6).
- **Renforcer l'ancrage temporel de l'agent** — élargir le banc de test sur les formulations relatives floues identifiées comme point faible (section 9.3.3), et évaluer si un prompt système plus directif ou un exemple few-shot suffit à corriger le comportement de Granite sur ce point précis.
- **Mémoire conversationnelle multi-session** — la `Session` actuelle (section 9.8) ne survit pas au redémarrage du script ; une persistance légère (SQLite, comme `EventStore`) permettrait de reprendre une conversation interrompue.